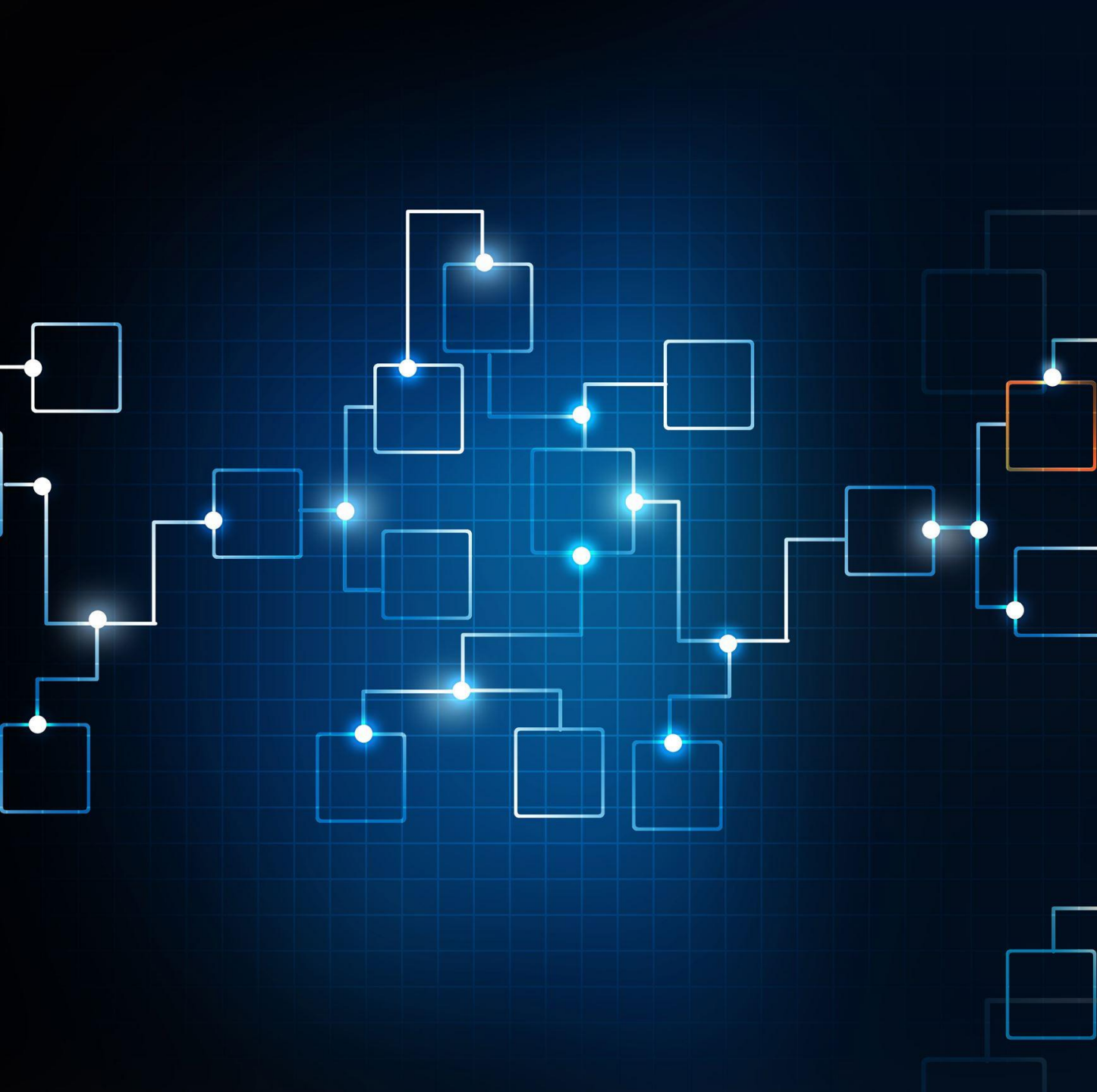




# Conceitos Básicos de Computação Paralela e Distribuída



AULA 3

PROF. PAULO BRESSAN



---

# Sumário

- Conceitos fundamentais
- Arquiteturas de sistemas paralelos
- Execução em Sistemas Paralelos

# Conceitos fundamentais

---



# Paralelismo como solução para problemas complexos

## Divisão de Tarefas

Problemas complexos são segmentados em tarefas menores para facilitar o processamento simultâneo.

## Execução Simultânea

Tarefas menores são executadas ao mesmo tempo para aumentar a eficiência do processamento.

## Redução do Tempo

O paralelismo diminui o tempo total necessário para resolver problemas complexos.

---

# Programação Paralela x Sequencial

Um processador paralelo é:

1. “Coleção de elementos de processamento

2. que se comunicam e cooperam entre si e

3. resolvem um problema mais rapidamente”

Algumas das questões relacionadas a cada uma das partes da definição acima são:

# I. “Coleção de elementos de processamento...”



qual o número ideal de elementos?



qual a potência e função de cada um?



qual a tecnologia adotada?



qual a quantidade de memória disponível?

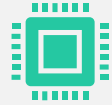


como é organizada a memória?



todos os elementos de processamento são iguais?

## 2. que se comunicam e cooperam entre si e...



como os elementos de processamento irão se comunicar?



como será implementado o sincronismo entre as tarefas?



como separar o problema em diversas tarefas?



como organizar a execução paralela, visando minimizar a comunicação e atingir um bom balanceamento?

### 3. resolvem um problema mais rapidamente”



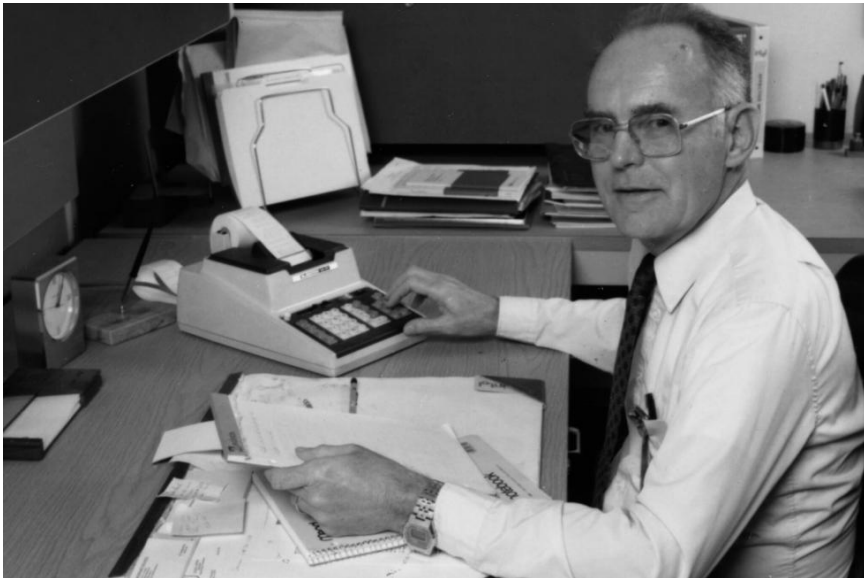
que tipos de problemas serão considerados?



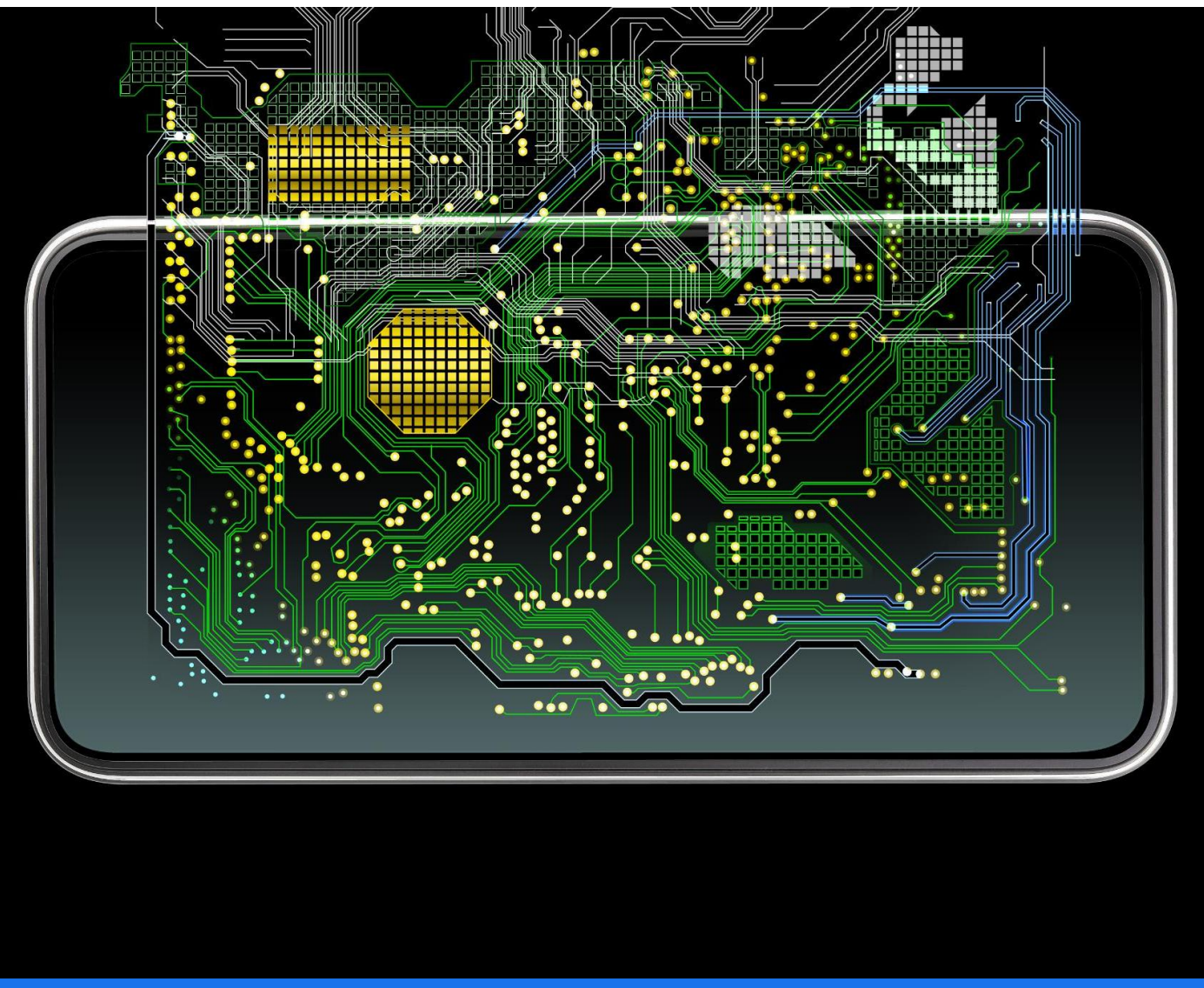
qual *speedup* a ser alcançado?

---

# Lei de Moore



- A **Lei de Moore** é uma observação empírica formulada por Gordon Moore, cofundador da Intel, em 1965 (e revisada em 1975).
- Ela afirma que o número de transistores em um circuito integrado (chip) dobra aproximadamente a cada dois anos, mantendo o custo de produção relativamente constante
- Na prática, significa aumento exponencial de poder computacional a um custo por transistor cada vez menor.
- A Lei de Moore não decorre de um princípio fundamental da natureza, é (ou foi?) uma **tendência tecnológica e econômica**.



# Limitações da computação sequencial e o fim da Lei de Moore

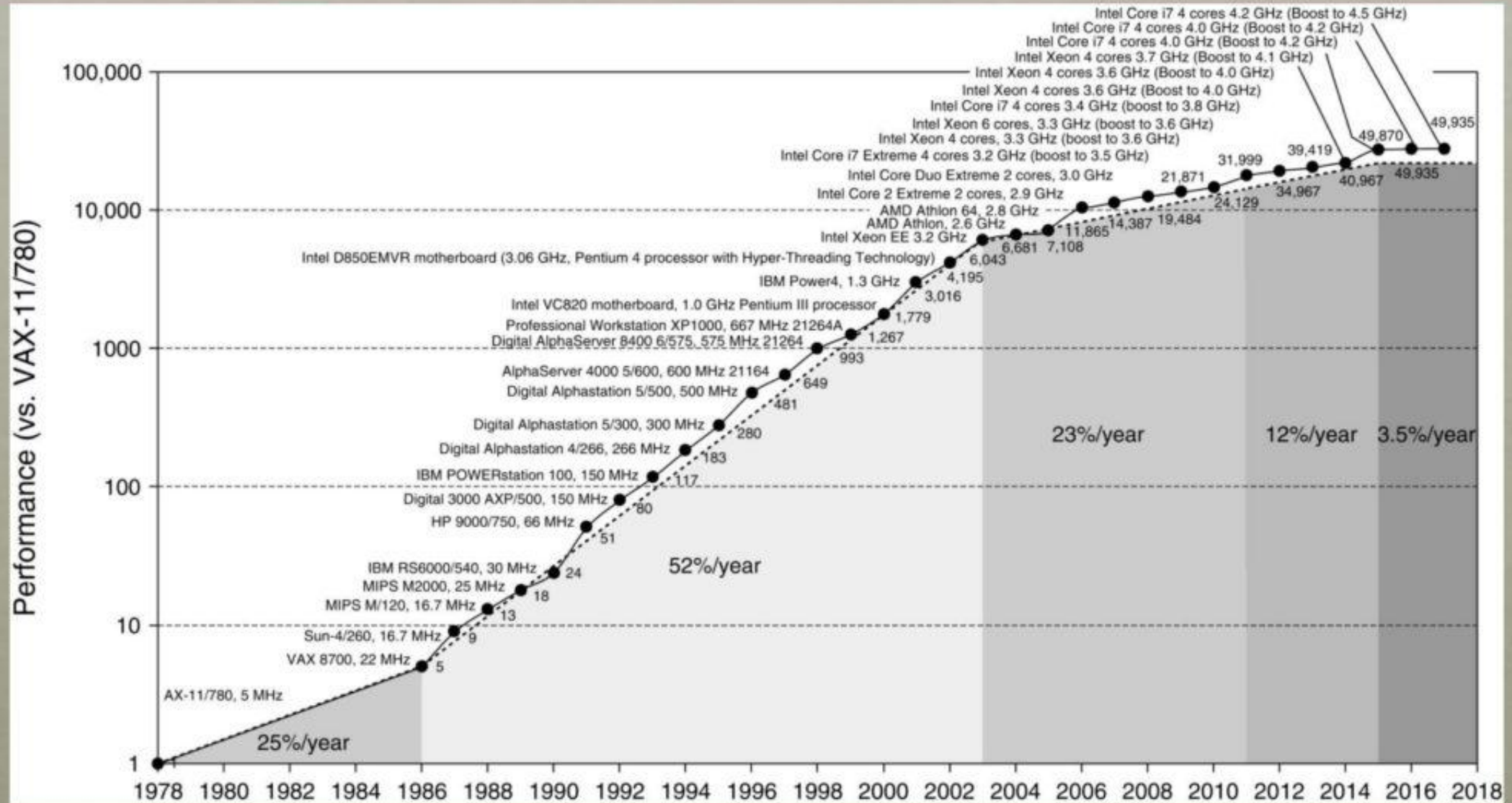
## Desempenho da Computação Sequencial

A computação sequencial enfrenta dificuldades para atender à demanda crescente por maior desempenho computacional.

## Desaceleração da Lei de Moore

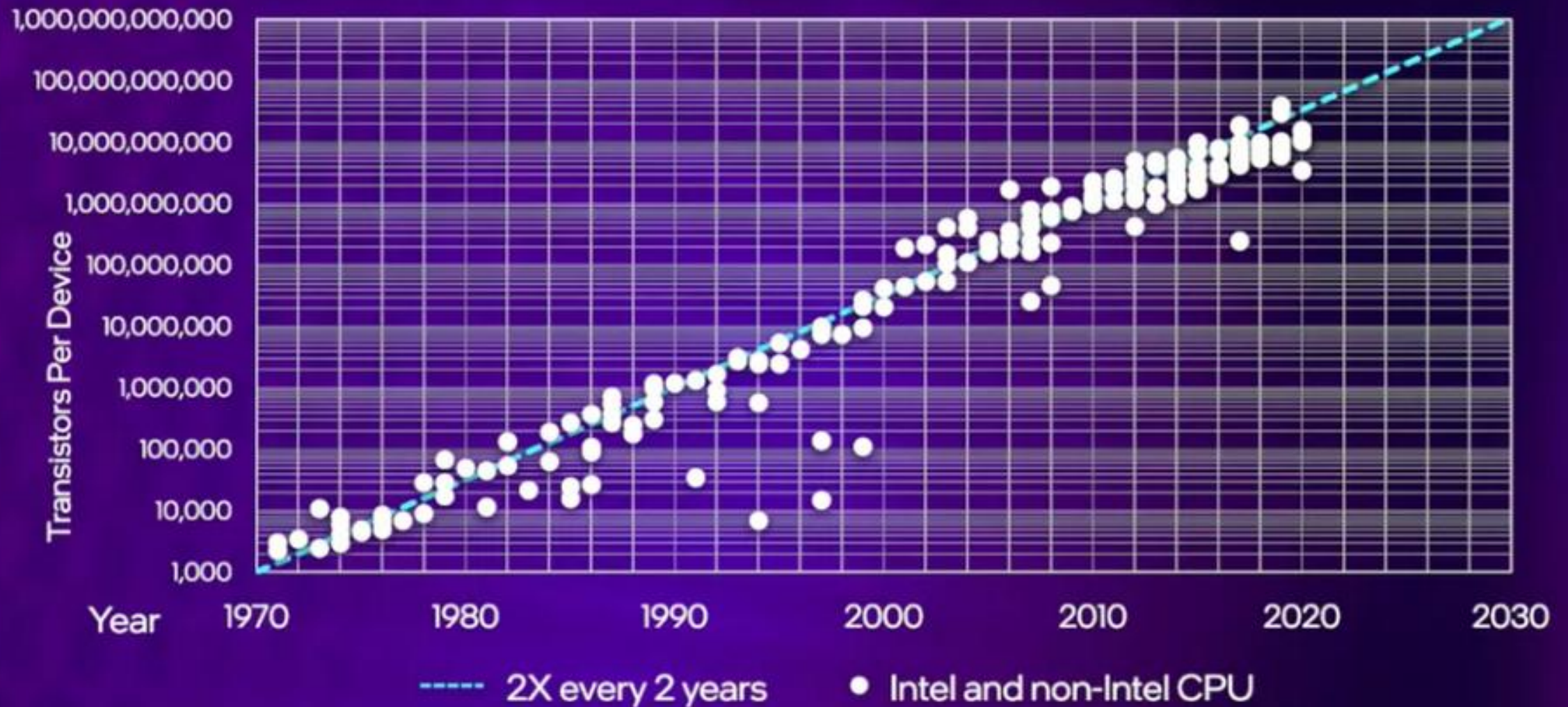
O avanço do hardware tradicional está desacelerando, mostrando limites previstos pela Lei de Moore.

# UNIPROCESSOR PERFORMANCE (SINGLE CORE)



# Moore's Law

is alive and well



# Tipos de paralelismo e níveis de execução

## Paralelismo de Dados

Paralelismo de dados processa múltiplos dados simultaneamente para acelerar tarefas repetitivas e grandes volumes de informações.

## Paralelismo de Tarefas

Paralelismo de tarefas executa diferentes tarefas ou processos em paralelo para otimizar o desempenho geral do sistema.

## Paralelismo em Pipeline

Paralelismo em pipeline divide a execução em etapas sequenciais sobrepostas para melhorar a eficiência da execução.



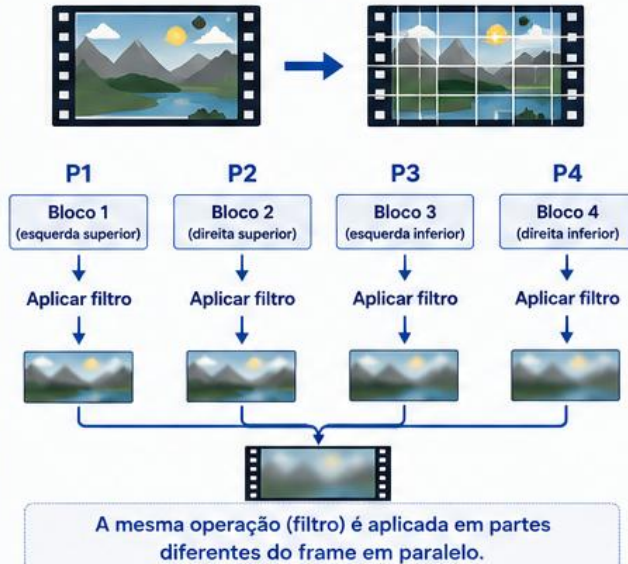
# COMPARAÇÃO ENTRE FORMAS DE PARALELISMO

## 1. PARALELISMO DE DADOS

A mesma operação é aplicada a diferentes partes dos dados.

### EXEMPLO: Processamento de vídeo

Aplicar o mesmo filtro (ex.: desfoque) em diferentes partes do mesmo frame.



### TEMPO

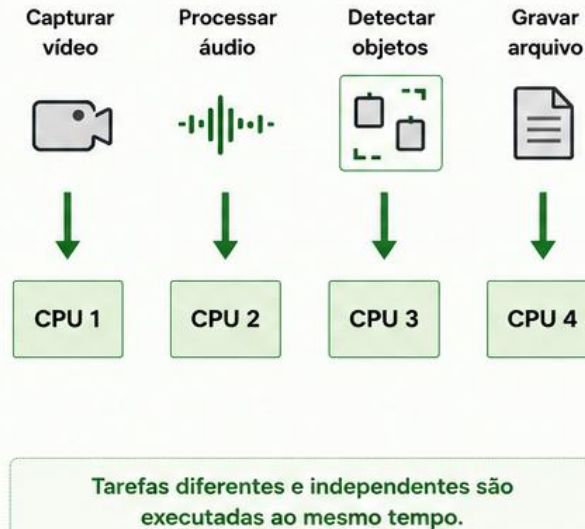


## 2. PARALELISMO DE TAREFAS

Tarefas diferentes são executadas simultaneamente.

### EXEMPLO: Processamento de vídeo

Tarefas diferentes do pipeline são executadas em paralelo.



### TEMPO

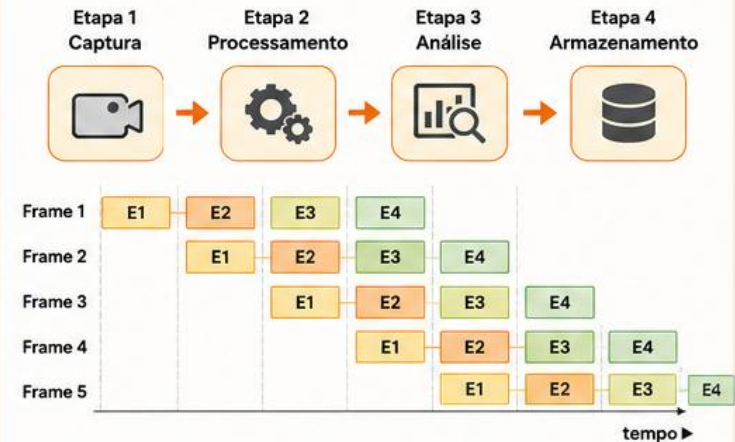


## 3. PARALELISMO EM PIPELINE

Diferentes etapas de um processo trabalham simultaneamente sobre diferentes itens.

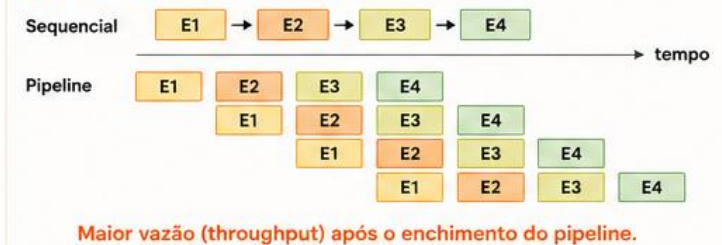
### EXEMPLO: Pipeline de processamento de vídeo

Frames diferentes avançam pelas etapas do pipeline. Em cada momento, cada etapa trabalha em um frame diferente.



Enquanto uma etapa processa um frame, as outras etapas processam frames diferentes.

### TEMPO



**PARALELISMO DE DADOS:**  
Mesma operação em dados diferentes.



**PARALELISMO DE TAREFAS:**  
Tarefas diferentes executadas simultaneamente.



**PARALELISMO EM PIPELINE:**  
Etapas diferentes trabalhando em paralelo sobre itens diferentes.

---

# Situação

## Exemplo:

# Processamento de Imagens



Temos 100 fotos e queremos aplicar 3 operações:



1. Converter para tons de cinza



2. Aplicar um filtro de nitidez

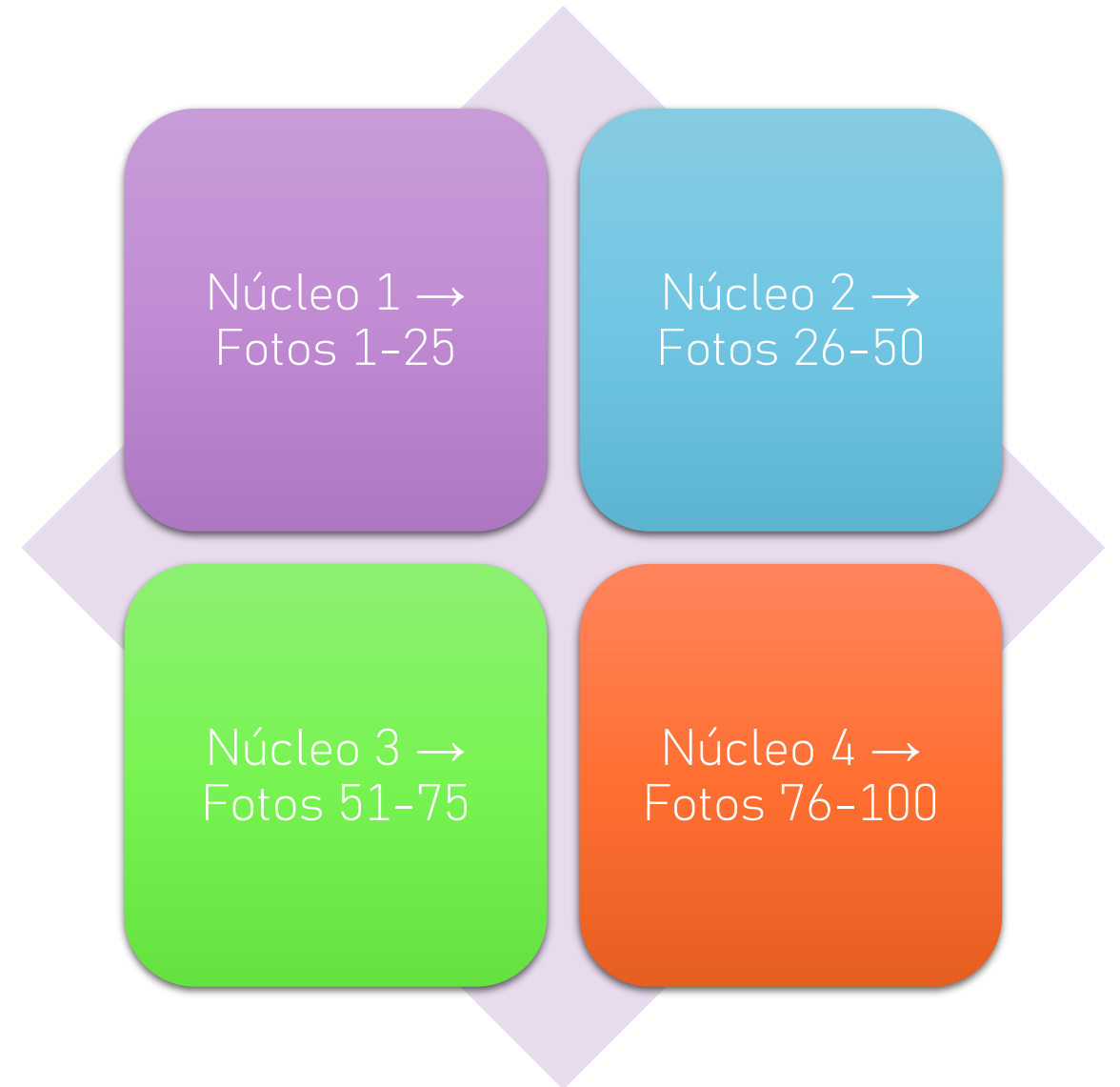


3. Redimensionar para 256x256

---

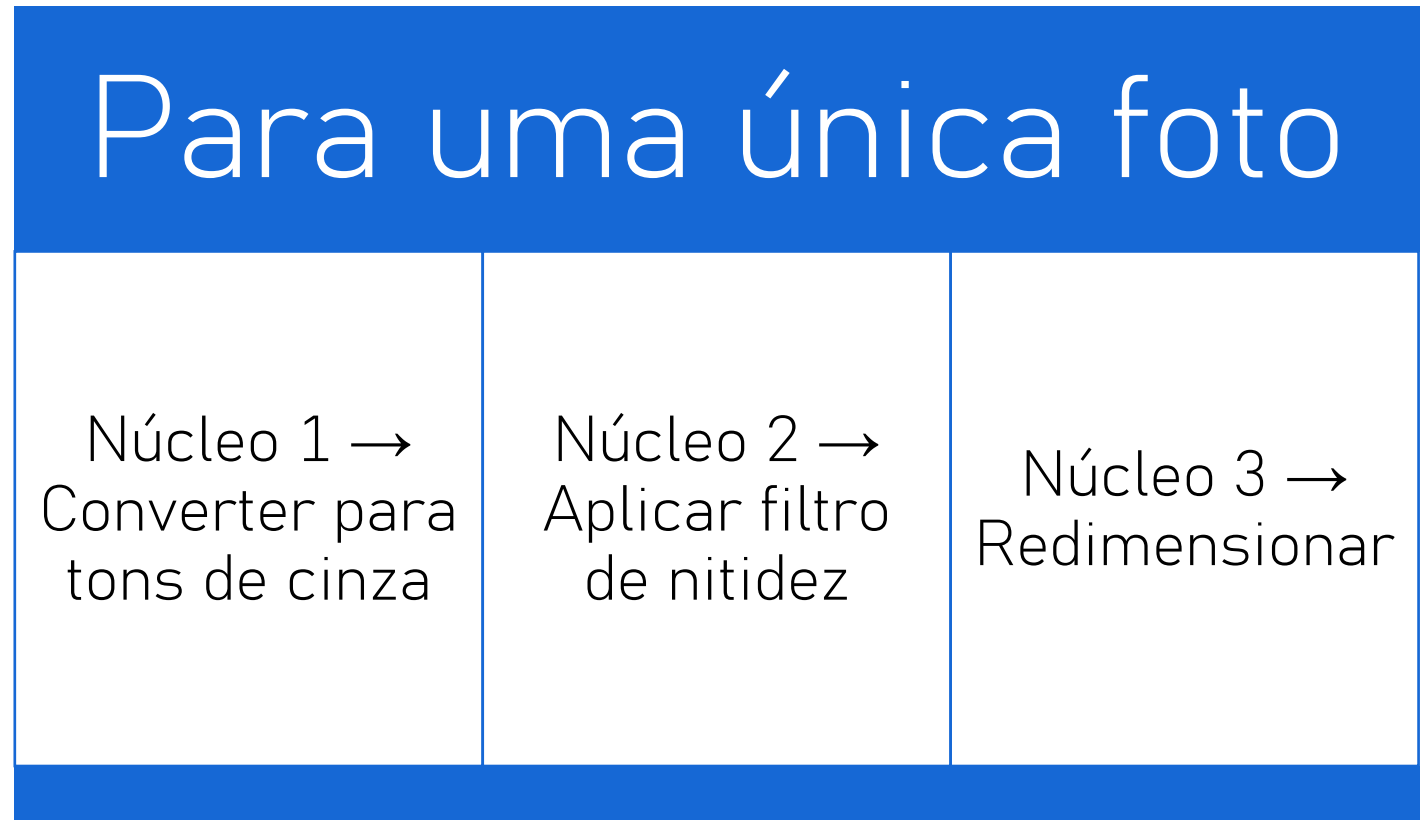
# Paralelismo de Dados

Mesma tarefa em muitos dados, cada núcleo executa a mesma operação em subconjuntos diferentes das fotos.



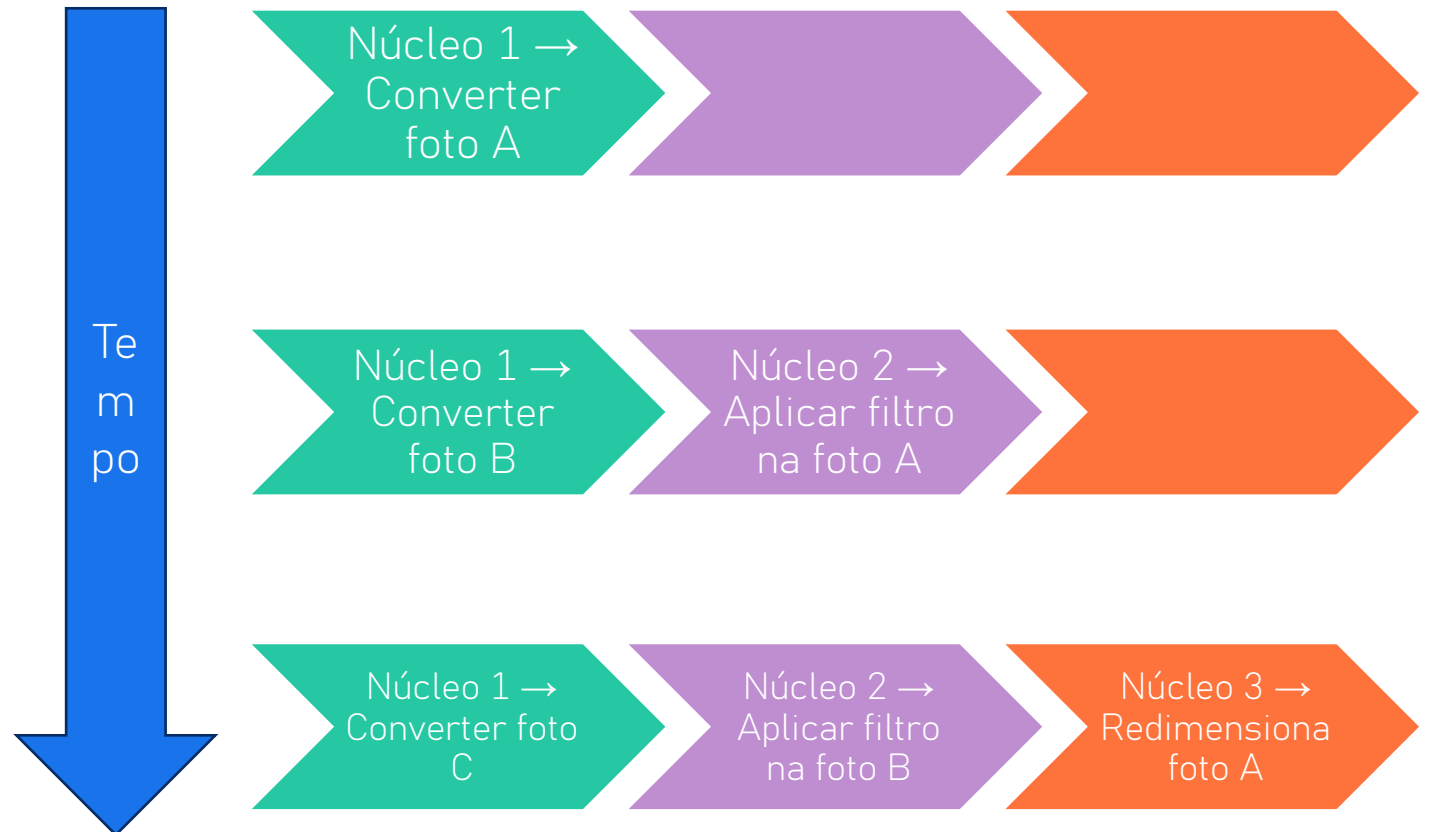
# Paralelismo de Tarefas

- Tarefas diferentes em paralelo, sobre o mesmo conjunto de dados.



# Paralelismo em Pipeline

Etapas sequenciais organizadas como linha de montagem.

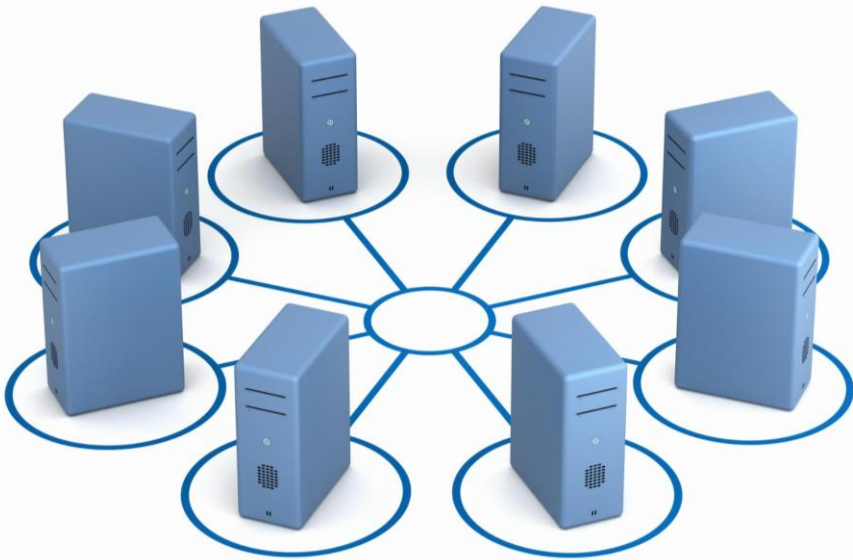


# Arquiteturas de sistemas paralelos

---

# Classificação de Flynn:

## SISD, SIMD, MISD, MIMD



### SISD - Fluxo único

SISD representa sistemas com um fluxo único de instrução e dados, caracterizando computadores sequenciais tradicionais.

### SIMD - Fluxo único, dados múltiplos

SIMD processa uma instrução com múltiplos dados simultaneamente, ideal para operações paralelas em vetores.

### MISD - Dados únicos, múltiplas instruções

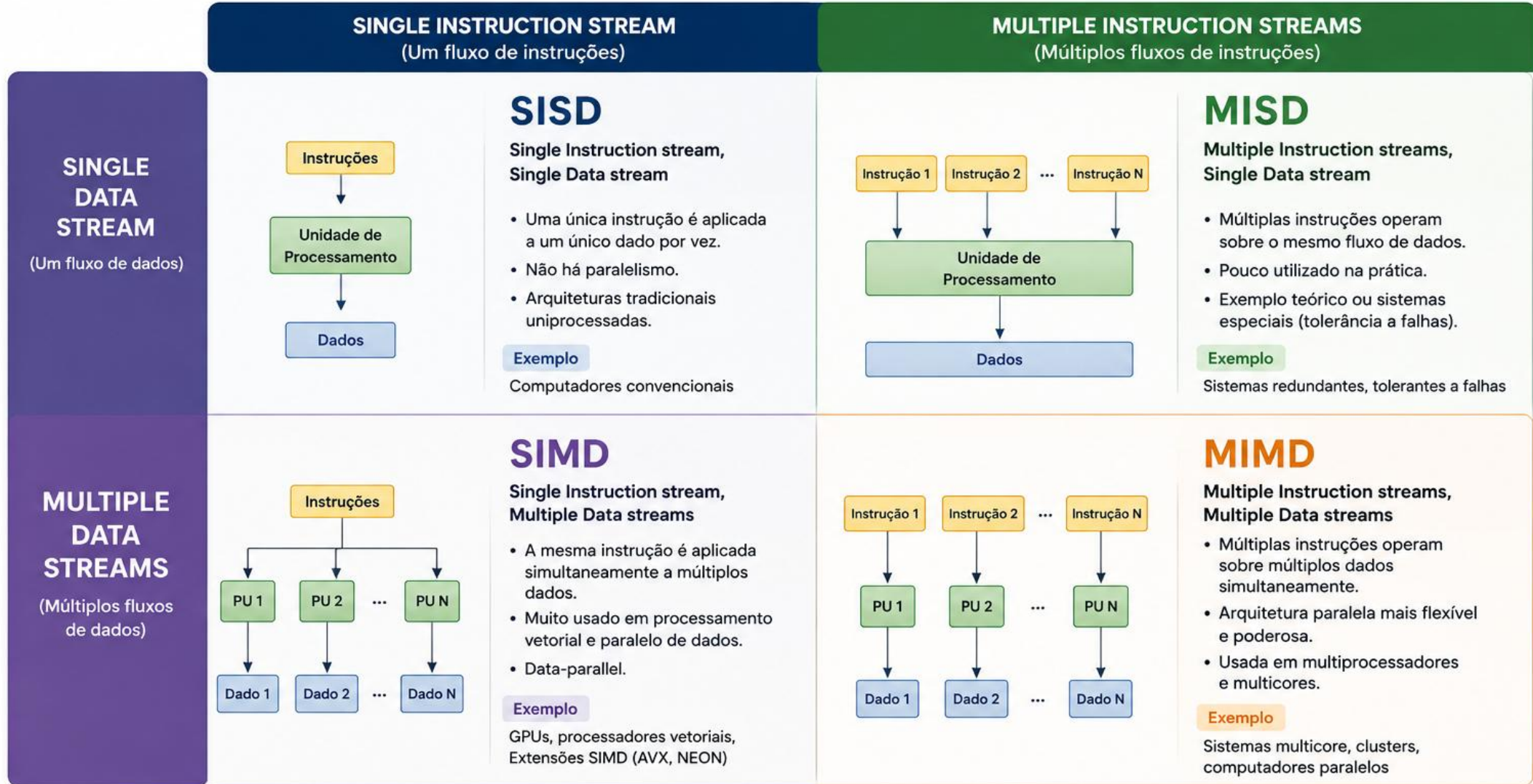
MISD possui múltiplos fluxos de instrução com dados únicos, sendo raro e usado em aplicações específicas.

### MIMD - Múltiplos fluxos

MIMD executa múltiplas instruções e dados simultaneamente, comum em sistemas paralelos e multiprocessadores.

# TAXONOMIA DE FLYNN

Classificação de arquiteturas paralelas de acordo com o número de fluxos de instruções e de dados



## LEGENDA

-  Fluxo(s) de Instruções
-  Fluxo(s) de Dados
-  Unidade(s) de Processamento

## RESUMO

- **SISD**: computação sequencial tradicional
- **MISD**: raro e pouco utilizado
- **SIMD**: paralelo de dados (mesma instrução, muitos dados)
- **MIMD**: paralelo de tarefas e dados (múltiplas instruções e dados)



A Taxonomia de Flynn ajuda a entender como as arquiteturas exploram o paralelismo, combinando fluxos de instruções e dados.

# MEMÓRIA COMPARTILHADA vs MEMÓRIA DISTRIBUÍDA

Comparação entre dois modelos fundamentais de arquitetura paralela

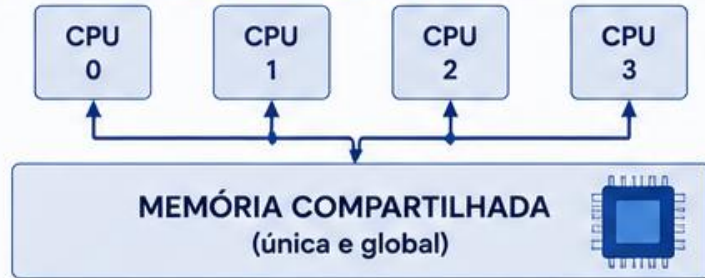
## MEMÓRIA COMPARTILHADA (Shared Memory)

VS

## MEMÓRIA DISTRIBUÍDA (Distributed Memory)

### IDEIA PRINCIPAL

Todos os processadores (ou núcleos) têm acesso a um **único espaço de memória global** compartilhado.



### CARACTERÍSTICAS

- Um único espaço de endereçamento.
- Comunicação implícita por leitura/escrita na memória.
- Sincronização com primitivas (locks, semáforos, barriers).
- Mais simples de programar.
- Escalabilidade limitada pelo barramento e pela coerência de cache.

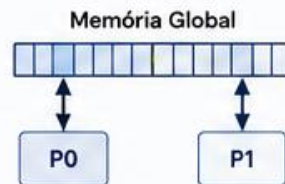
### EXEMPLOS TÍPICOS

- SMP (Symmetric Multiprocessors)
- Multicore / Manycore de um mesmo nó
- OpenMP, Pthreads, TBB

### COMUNICAÇÃO

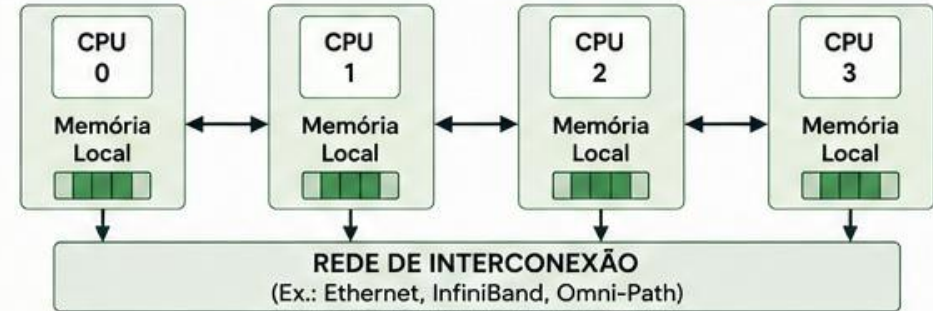
Acesso direto à memória compartilhada.

```
// Exemplo: escrita de P0, leitura de P1
A[100] = 42;      // P0 escreve
...
valor = A[100];   // P1 lê
```



### IDEIA PRINCIPAL

Cada processador tem sua **própria memória local**. Processadores se comunicam trocando **mensagens**.



### CARACTERÍSTICAS

- Múltiplos espaços de endereçamento.
- Comunicação explícita por troca de mensagens (MPI, send/receive).
- Maior escalabilidade.
- Maior complexidade de programação.
- Não há coerência de cache entre nós.

### EXEMPLOS TÍPICOS

- Clusters de computadores
- Supercomputadores
- MPI** (Message Passing Interface)

### COMUNICAÇÃO

Troca explícita de mensagens.

```
// Exemplo: P0 envia, P1 recebe
MPI_Send(&valor, 1, MPI_INT, 1, TAG, COMM);
...
MPI_Recv(&valor, 1, MPI_INT, 0, TAG, COMM, &status);
```



| COMPARAÇÃO RÁPIDA         |                               |                                 |
|---------------------------|-------------------------------|---------------------------------|
| Aspecto                   | Memória Compartilhada         | Memória Distribuída             |
| Espaço de endereçamento   | Único e global                | Múltiplos (um por processo)     |
| Comunicação               | Leitura/escrita na memória    | Troca de mensagens (MPI)        |
| Sincronização             | Primitivas de sincronização   | Sincronização por mensagens     |
| Facilidade de programação | Mais fácil                    | Mais complexa                   |
| Escalabilidade            | Limitada (dezenas de núcleos) | Alta (centenas/milhares de nós) |
| Custo                     | Menor (um único nó)           | Maior (múltiplos nós + rede)    |
| Exemplos                  | OpenMP, Pthreads              | MPI, GASNet, UPC                |

| QUANDO USAR?                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>USE MEMÓRIA COMPARTILHADA QUANDO:</b></p> <ul style="list-style-type: none"> <li>✓ O problema roda em um único servidor ou máquina.</li> <li>✓ O número de núcleos é moderado.</li> <li>✓ Você precisa de programação mais simples.</li> <li>✓ A comunicação é frequente e fina.</li> </ul>  | <p><b>USE MEMÓRIA DISTRIBUÍDA QUANDO:</b></p> <ul style="list-style-type: none"> <li>✓ O problema é muito grande para um único nó.</li> <li>✓ É necessário escalar para muitos nós.</li> <li>✓ A comunicação pode ser agrupada (menos frequente).</li> <li>✓ Busca-se máximo desempenho em HPC.</li> </ul>  |

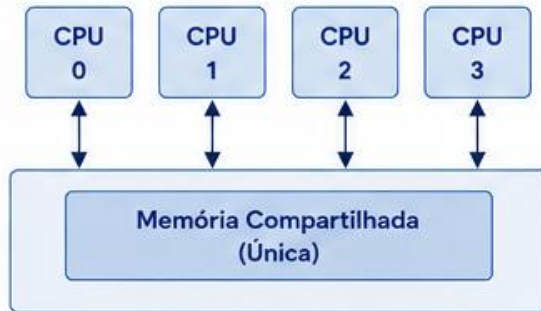
# MODELOS DE MEMÓRIA EM ARQUITETURAS PARALELAS

Comparação entre UMA, NUMA, Distributed Memory e Hybrid Memory

## UMA

(Uniform Memory Access)

Todos os processadores têm acesso **uniforme** (mesma latência) a toda a memória compartilhada.



### Características

- Uma única memória física global.
- Mesmo tempo de acesso para todos.
- Hardware mais simples (geralmente SMP).
- Escalabilidade limitada pelo barramento de memória.

### Exemplo típico

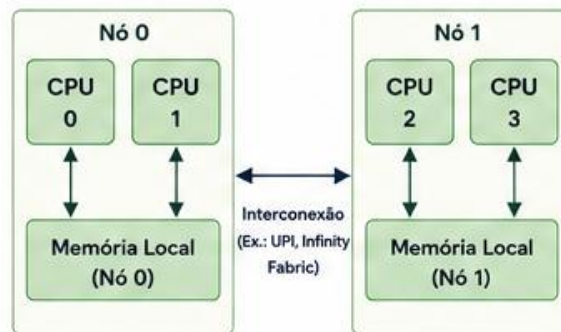


Servidores SMP de pequeno a médio porte.

## NUMA

(Non-Uniform Memory Access)

A memória é distribuída entre nós, mas todo processador pode acessar a memória de qualquer nó (com **latência variável**).



### Características

- Memória fisicamente distribuída por nós.
- Acesso local é mais rápido que acesso remoto.
- Todos os processadores compartilham o espaço de endereçamento global.
- Escalabilidade melhor que UMA.

### Exemplo típico

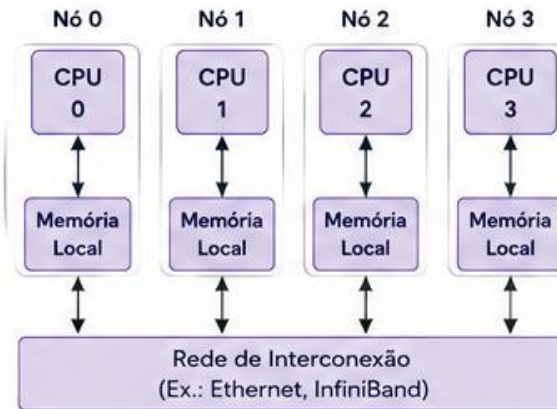


Servidores de grande porte (com Intel Xeon Scalable, AMD EPYC).

## DISTRIBUTED MEMORY

(Memória Distribuída)

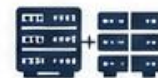
Cada processador (ou nó) tem sua própria memória local. Não há **espaço de memória compartilhado** globalmente.



### Características

- Memória privada para cada nó.
- Comunicação explícita por troca de mensagens (MPI, Send/Receive).
- Alta escalabilidade e baixo custo por nó.
- Maior complexidade de programação.

### Exemplo típico

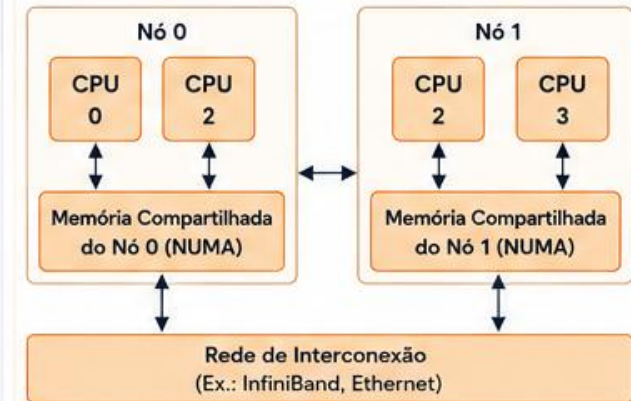


Clusters de computadores, HPC (supercomputadores).

## HYBRID MEMORY

(Memória Híbrida)

Combina memória **compartilhada** dentro de cada nó (NUMA) com memória **distribuída** entre nós.



### Características

- Memória compartilhada dentro do nó (NUMA).
- Memória distribuída entre nós.
- Explora paralelismo com threads (OpenMP) e com mensagens (MPI).
- Melhor desempenho e escalabilidade.

### Exemplo típico



Supercomputadores modernos, clusters de alto desempenho.

| <br>COMPARAÇÃO<br>RÁPIDA | Modelo      | Memória Global      | Acesso Remoto       | Programação                           | Escalabilidade | Latência                   |
|-----------------------------------------------------------------------------------------------------------|-------------|---------------------|---------------------|---------------------------------------|----------------|----------------------------|
|                                                                                                           | UMA         | Sim (única)         | Não existe          | Mais simples (threads)                | Baixa a média  | Uniforme                   |
|                                                                                                           | NUMA        | Sim (compartilhada) | Existe (mais lenta) | Threads (atenção à afinação de dados) | Média a alta   | Não uniforme               |
|                                                                                                           | Distributed | Não                 | N/A (mensagens)     | Mensagens (MPI)                       | Muito alta     | Comunicação (explícita)    |
|                                                                                                           | Hybrid      | Sim (por nó)        | Sim (entre nós)     | Híbrida (MPI + threads)               | Muito alta     | Não uniforme + comunicação |

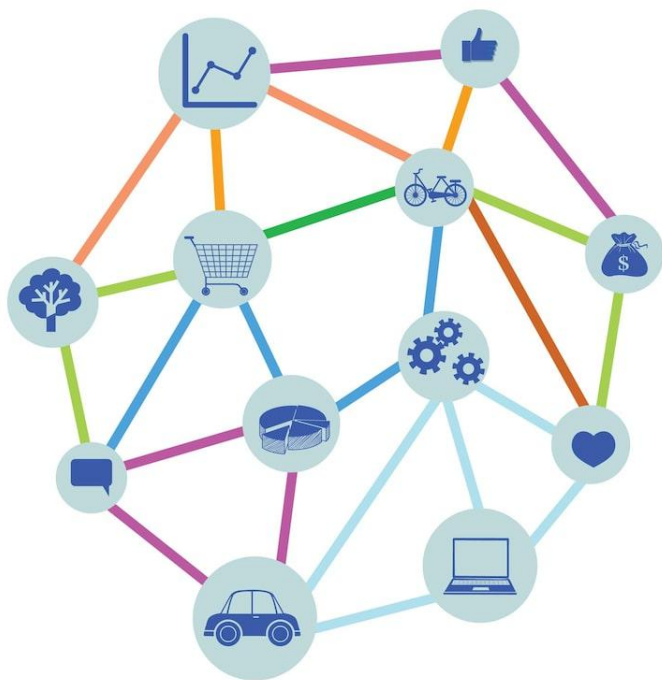


### QUANDO USAR?

- **UMA:** sistemas menores, até dezenas de CPUs.
- **NUMA:** servidores grandes, aplicações com muitos threads e dados compartilhados.
- **Distributed Memory:** problemas muito grandes, clusters, supercomputadores.
- **Hybrid Memory:** aplicações que precisam de alta escalabilidade e desempenho (HPC moderno).

---

# Redes de interconexão



Redes de interconexão são a infraestrutura de comunicação que conecta processadores, memórias e/ou nós de um sistema, determinando como as unidades de processamento trocam dados entre si.

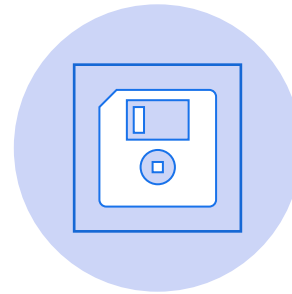
É um tópico central em MIMD (multiprocessadores e clusters), já que o desempenho do paralelismo depende tanto da capacidade de cálculo quanto da eficiência da comunicação.

---

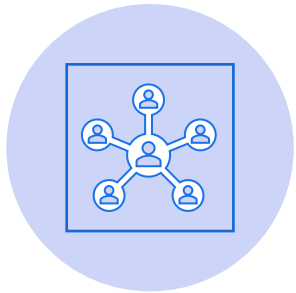
# Redes de Interconexão



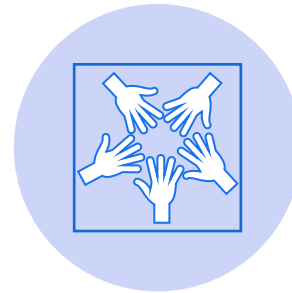
Função: comunicação entre processadores e memória.



Diferença entre sistemas de memória compartilhada e distribuída.



Estáticas (diretas): conexões fixas.



Dinâmicas (indiretas): uso de comutadores.

---

# Classificação Básica

## 1. Quanto à conexão física

- **Redes estáticas (diretas):** cada nó tem conexões fixas e ponto-a-ponto com seus vizinhos. Usadas em sistemas de memória distribuída (clusters).
- **Redes dinâmicas (indiretas):** usam switches/comutadores para estabelecer caminhos entre nós, permitindo reconfiguração. Comuns em sistemas de memória compartilhada.

# Classificação Básica

## 2. Quanto à topologia

*Redes estáticas:*

- **Barramento (bus):** todos os nós compartilham um único canal. Simples e barato, mas gargalo de contenção cresce com o número de nós.
- **Anel (ring):** cada nó conecta-se a dois vizinhos, formando um ciclo. Baixo custo, mas latência cresce linearmente com o tamanho.
- **Malha/grade (mesh):** nós organizados em grade 2D ou 3D, conectados aos vizinhos adjacentes. Bom equilíbrio custo/desempenho, usado em supercomputadores.

# Classificação Básica

## 2. Quanto à topologia

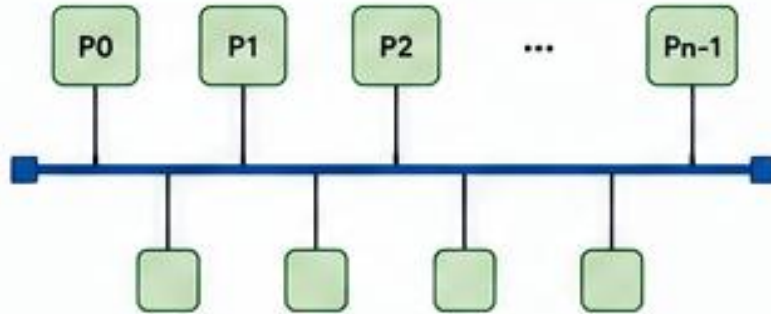
*Redes estáticas:*

- **Toro (torus)**: malha com wraparound (bordas conectadas), reduzindo o diâmetro da rede.
- **Árvore (tree)**: hierarquia com raiz e folhas; pode sofrer gargalo perto da raiz (mitigado pela "fat tree", onde os enlaces ficam mais largos perto do topo).
- **Hipercubo (hypercube)**: cada nó conecta-se a  $\log_2(N)$  vizinhos que diferem em um bit no endereço binário. Diâmetro pequeno, boa escalabilidade, mas custo de fiação alto.
- **Totalmente conectada (fully connected)**: cada nó liga-se a todos os outros. Latência mínima, mas custo  $O(N^2)$  — inviável para N grande.

# REDES DE INTERCONEXÃO: TOPOLOGIAS ESTÁTICAS

*Topologias usadas em arquiteturas paralelas e sistemas de alto desempenho*

## 1. BARRAMENTO (BUS)

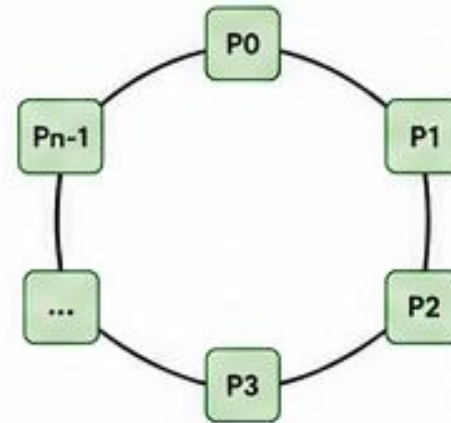


Todos os nós compartilham um único meio de comunicação.

### Características

- ✓ Simples e barato
- ✓ Fácil de implementar
- ✗ Baixa escalabilidade
- ✗ Ponto único de falha (barramento)
- i Alto contencimento

## 2. ANEL (RING)



Cada nó conectado a dois vizinhos, formando um anel fechado.

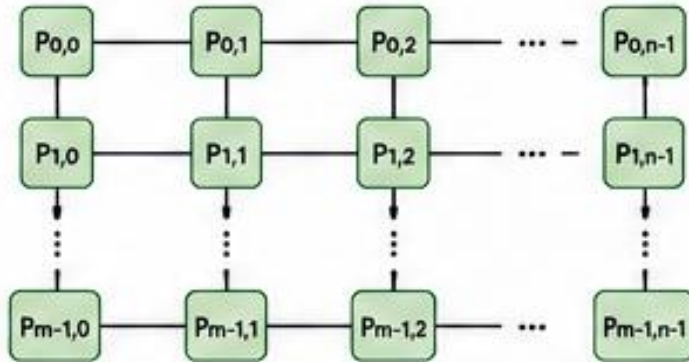
### Características

- ✓ Simples e de baixo custo
- ✓ Tráfego previsível
- ✗ Latência cresce com o número de nós
- ✗ Uma falha pode parar a rede (sem redundância)
- i Usada em redes circulares e tokens

# REDES DE INTERCONEXÃO: TOPOLOGIAS ESTÁTICAS

Topologias usadas em arquiteturas paralelas e sistemas de alto desempenho

## 3. MALHA / GRADE (MESH 2D)

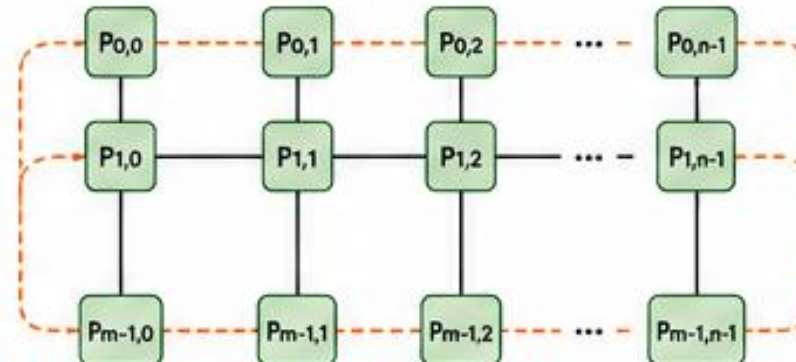


Nós organizados em uma grade 2D.  
Cada nó ligado aos seus vizinhos (N, S, L, O).

### Características

- ✓ Alta escalabilidade
- ✓ Rotas múltiplas possíveis
- ✗ Diâmetro cresce com  $\sqrt{N}$
- ✗ Custo de hardware moderado
- i Muito usada em clusters e supercomputadores

## 4. TORO (2D TORUS)



Malha 2D com conexões wrap-around  
(conexões fechadas nas bordas).

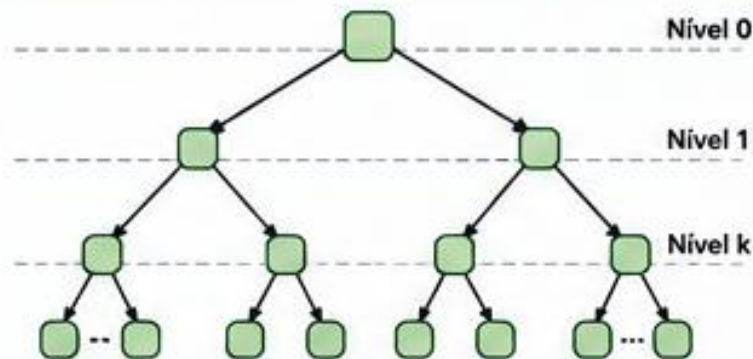
### Características

- ✓ Menor diâmetro que a malha
- ✓ Balanceamento de carga melhor
- ✓ Mais caminhos alternativos
- ✗ Mais complexa que a malha
- i Muito usada em HPC (ex.: Blue Gene)

# REDES DE INTERCONEXÃO: TOPOLOGIAS ESTÁTICAS

Topologias usadas em arquiteturas paralelas e sistemas de alto desempenho

## 5. ÁRVORE (TREE)

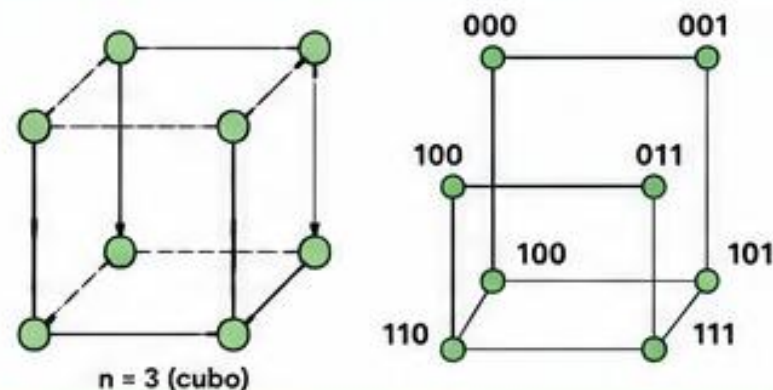


Hierárquica: nós organizados em níveis com um ou mais caminhos para a raiz.

### Características

- ✓ Escalável
- ✓ Fácil agregação hierárquica
- ✗ Alto diâmetro (latência maior)
- ✗ Congestionamento próximo à raiz
- i Usada em redes fat-tree, cluster hierárquico

## 6. HIPERCUBO (k-ARY n-CUBE)



Cada nó tem  $n$  vizinhos ( $2^n$  nós no total).  
Conexões baseadas em diferença de 1 bit nos endereços.

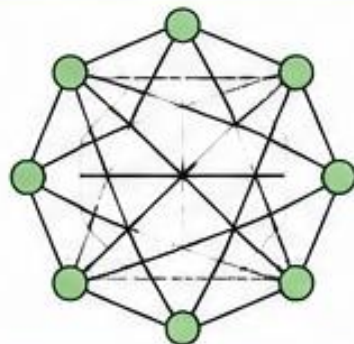
### Características

- ✓ Baixa latência média
- ✓ Muitos caminhos alternativos
- ✓ Simétrico e regular
- ✗ Custo de hardware cresce com  $n$
- i Usado em supercomputadores (ex.: Intel Paragon)

# REDES DE INTERCONEXÃO: TOPOLOGIAS ESTÁTICAS

Topologias usadas em arquiteturas paralelas e sistemas de alto desempenho

## 7. TOTALMENTE CONECTADA (CLIQUE)



Todos os nós conectados a todos os outros nós.

### Características

- ✓ Máxima largura de banda
- ✓ Menor latência (1 salto)
- ✗ Custo muito alto:  $N(N-1)/2$  enlaces
- Impraticável para grande número de nós
- i Usada apenas em sistemas pequenos

### LEGENDA

- Nó (processador)
- Enlace de comunicação
- Conexão wrap-around

### COMPARAÇÃO GERAL

|                      |   |                      |
|----------------------|---|----------------------|
| Menor custo          | → | Maior custo          |
| Menor latência       | → | Maior latência       |
| Menor diâmetro       | → | Maior diâmetro       |
| Menor escalabilidade | → | Maior escalabilidade |

### ESCOLHA DA TOPOLOGIA



Depende do número de nós, custo do hardware, tráfego de comunicação, escalabilidade e tolerância a falhas desejadas.

# Classificação Básica

## 2. Quanto à topologia

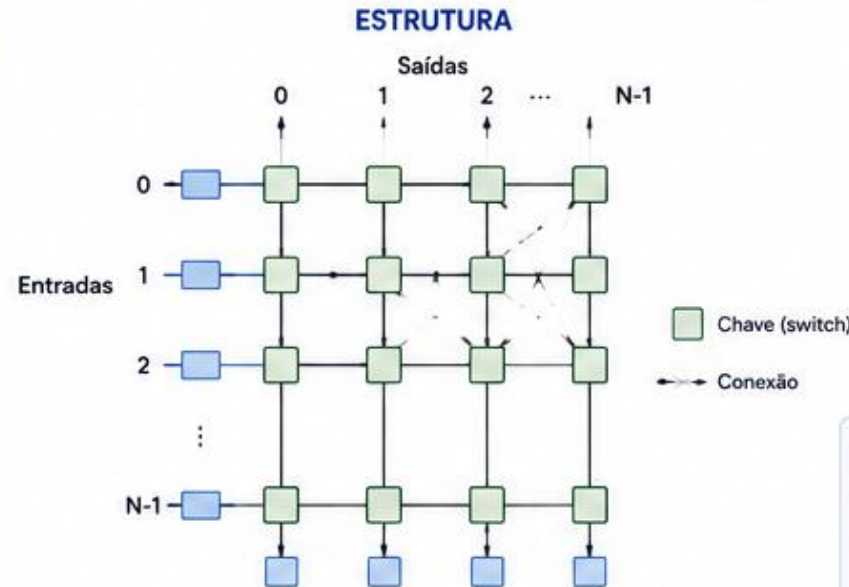
*Redes dinâmicas:*

- **Crossbar**: matriz de switches conectando cada entrada a qualquer saída sem bloqueio. Excelente desempenho, mas custo  $O(N^2)$  em hardware.
- **Redes multiestágio (multistage)**: como a rede Omega/Butterfly, usam múltiplos estágios de switches menores para aproximar o desempenho de um crossbar a custo menor ( $O(N \log N)$ ).

# REDES DE INTERCONEXÃO: CROSSBAR E MULTIESTÁGIO

Topologias de alta largura de banda usadas em arquiteturas paralelas e sistemas de alto desempenho

## 1. CROSSBAR (MALHA COMPLETA DE CHAVES)



### COMO FUNCIONA

- Cada entrada pode se conectar a qualquer saída através de uma chave.
- Permite múltiplas comunicações simultâneas, desde que não usem a mesma saída.

### EXEMPLO (N = 4)

Conexões simultâneas possíveis:

- 0 → 1, 1 → 3, 2 → 0, 3 → 2 ✓
- 0 → 1, 1 → 1, 2 → 1, 3 → 1 ✗  
(conflito na saída 1)

### CARACTERÍSTICAS

- ✓ Largura de banda muito alta
- ✓ Latência muito baixa e constante
- ✓ Permite N comunicações simultâneas (uma por saída)
- ✗ Custo cresce com  $N^2$  ( $N \times N$  chaves)
- ✗ Escalabilidade limitada pelo custo e complexidade

### APLICAÇÕES

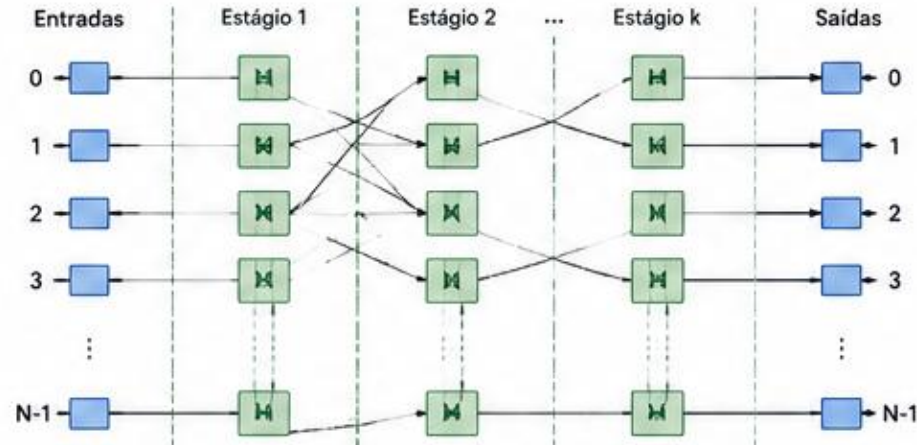
- ✎ Sistemas de comutação de alta velocidade
- ✎ Multiprocessadores de pequeno e médio porte
- ✎ Backplanes de roteadores e switches
- ✎ Interconexão em FPGAs e NoCs

# REDES DE INTERCONEXÃO: CROSSBAR E MULTISTÁGIO

Topologias de alta largura de banda usadas em arquiteturas paralelas e sistemas de alto desempenho

## 2. REDES MULTISTÁGIO (MULTI-STAGE NETWORKS)

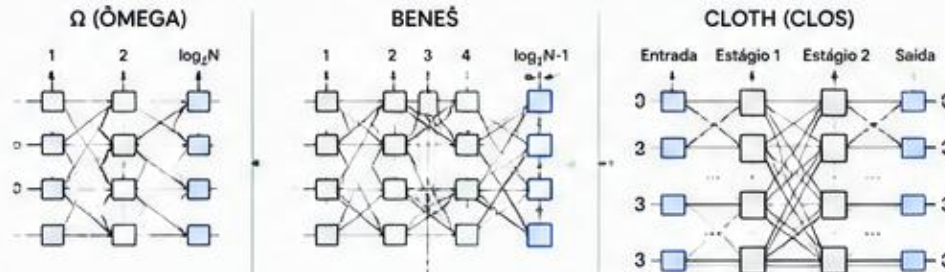
### ESTRUTURA



### COMO FUNCIONA

- A comunicação entre uma entrada e uma saída é feita através de um caminho formado por várias chaves em estágios.
- O roteamento escolhe, em cada estágio, a próxima direção até alcançar a saída desejada.

### EXEMPLOS DE REDES MULTISTÁGIO



### CARACTERÍSTICAS

- ✓ Escalabilidade muito alta
- ✓ Menor custo que o crossbar
- ✓ Requer algoritmo de roteamento
- ✗ Maior latência (múltiplos estágios)
- ✗ Pode haver bloqueio (contention) dependendo do tipo de rede

### APLICAÇÕES



Interconexão em supercomputadores (ex.: NoCs, clusters)



Switches de grande porte (telecomunicações)



Redes de comutação em sistemas paralelos

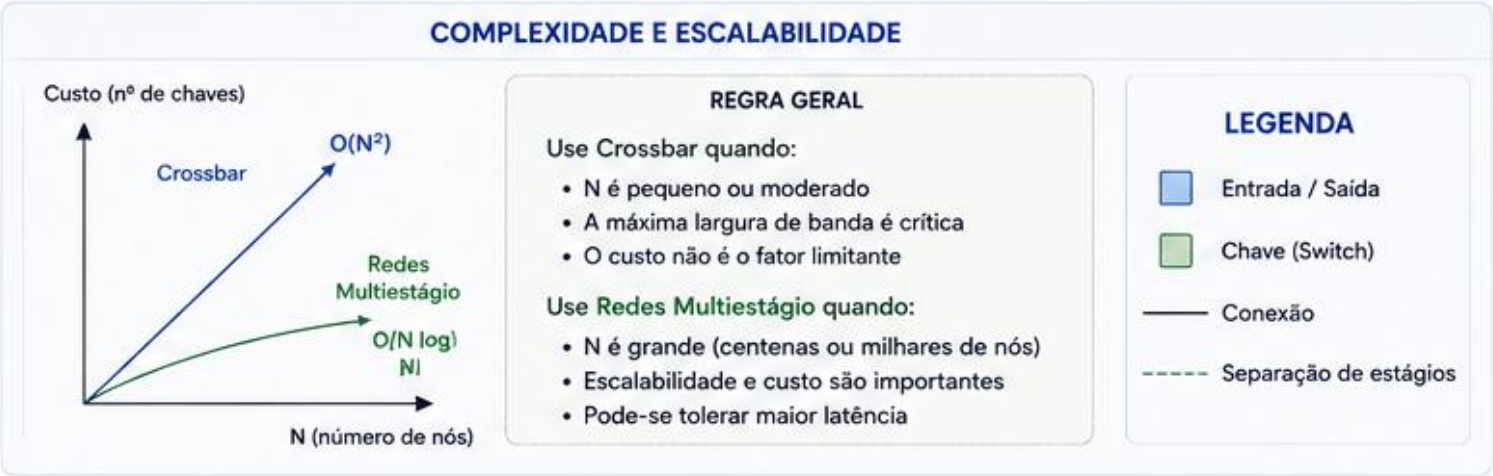


Sistemas onde o número de nós é muito grande

# REDES DE INTERCONEXÃO: CROSSBAR E MULTISTÁGIO

Topologias de alta largura de banda usadas em arquiteturas paralelas e sistemas de alto desempenho

| COMPARAÇÃO RÁPIDA        |                                            |                                              |
|--------------------------|--------------------------------------------|----------------------------------------------|
| Aspecto                  | Crossbar                                   | Redes Multiestágio                           |
| Conectividade            | Direta (qualquer entrada a qualquer saída) | Indireta (através de estágios)               |
| Largura de banda         | Muito alta                                 | Alta (depende do número de estágios e links) |
| Latência                 | Muito baixa                                | Maior (passa por vários estágios)            |
| Custo                    | $O(N^2)$                                   | $O(N \log N)$ ou $O(N \log^2 N)$             |
| Escalabilidade           | Baixa                                      | Alta                                         |
| Complexidade de controle | Baixa                                      | Maior (roteamento entre estágios)            |
| Uso típico               | Sistemas menores / alta performance        | Sistemas grandes / supercomputadores         |





# Critérios de Avaliação

- **Diâmetro:** maior distância (em saltos) entre dois nós quaisquer — impacta a latência no pior caso.
- **Largura de bissecção (bisection width):** número mínimo de enlaces que, removidos, dividem a rede em duas partes iguais — indica a capacidade de comunicação global.
- **Grau do nó:** número de conexões por nó — afeta custo e complexidade de roteamento.
- **Custo:** geralmente proporcional ao número total de enlaces.

# CRITÉRIOS DE AVALIAÇÃO EM REDES DE INTERCONEXÃO

Métricas fundamentais para comparar topologias de redes de interconexão em arquiteturas paralelas

## 1 DIÂMETRO (D)

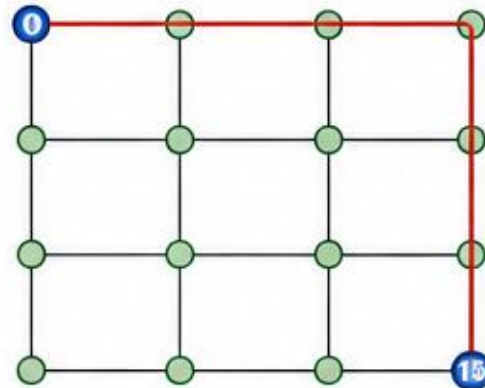
Maior distância entre quaisquer dois nós da rede.

### DEFINIÇÃO

É o maior número mínimo de saltos (hops) necessários para ir de um nó a outro.

### EXEMPLO

Rede em Malha 2D 4×4



Caminho mínimo entre 0 e 15 = 6 saltos  
Logo, o diâmetro  $D = 6$

### IMPORTÂNCIA

Indica o pior caso de latência (número máximo de saltos) entre dois nós da rede.

## 2 LARGURA DA BISSECÇÃO (B)

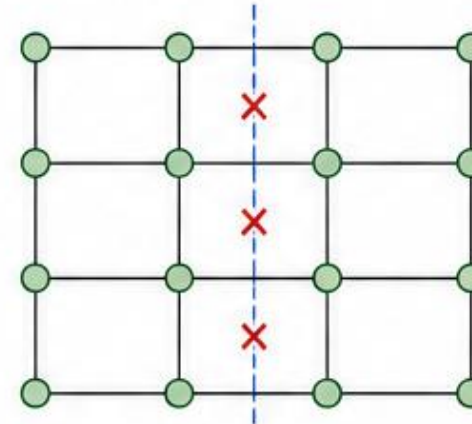
Menor número de enlaces que, quando removidos, particionam a rede em duas partes com o mesmo tamanho.

### DEFINIÇÃO

É a capacidade mínima de separação da rede. Quanto maior, maior o paralelismo possível com menor congestionamento.

### EXEMPLO

Malha 4×4 – Bissecção Vertical



A menor bissecção corta 4 enlaces  
Logo,  $B = 4$

### IMPORTÂNCIA

Limita a largura de banda agregada entre duas metades da rede. Importante para evitar gargalos de comunicação.

# CRITÉRIOS DE AVALIAÇÃO EM REDES DE INTERCONEXÃO

Métricas fundamentais para comparar topologias de redes de interconexão em arquiteturas paralelas

## 3 GRAU DO NÓ (g)

Número de enlaces (conexões) incidentes em um nó.

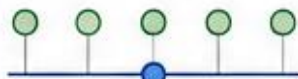
### DEFINIÇÃO

É o número de vizinhos diretos que um nó possui. Em redes regulares, todos os nós têm o mesmo grau.

### EXEMPLO

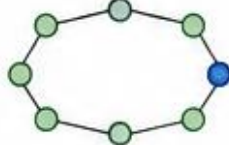
Topologias e graus (g)

Barramento



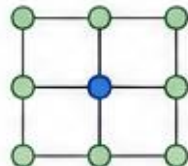
$$g_{\text{extremo}} = 1 \quad g_{\text{interno}} = 2$$

Anel



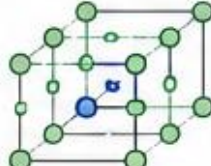
$$g = 2$$

Malha 2D (4-vizinhos)



$$g_{\text{canto}} = 2 \quad g_{\text{borda}} = 3 \quad g_{\text{interno}} = 4$$

Cubo 3D (Hiper cubo)



$$g = 3$$

### IMPORTÂNCIA

Afeta o custo (número de enlaces/switches), a complexidade do roteamento e o grau de paralelismo que cada nó pode sustentar.

## 4 CUSTO (C)

Recursos físicos necessários para construir a rede.

### DEFINIÇÃO

É o número total de enlaces (e/ou switches) necessários para implementar a topologia.

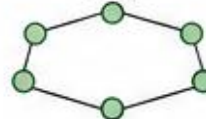
### EXEMPLOS - N nós

Barramento



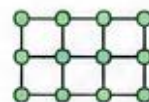
$$C = N - 1$$

Anel



$$C = N$$

Malha 2D ( $\sqrt{N} \times \sqrt{N}$ )



$$C = 2N - 2\sqrt{N}$$

Cubo n-D ( $N = 2^n$ )



$$C = \frac{nN}{2}$$

### IMPORTÂNCIA

Determina o investimento em hardware, consumo de área, energia e complexidade de implementação.

# CRITÉRIOS DE AVALIAÇÃO EM REDES DE INTERCONEXÃO

Métricas fundamentais para comparar topologias de redes de interconexão em arquiteturas paralelas

## COMPARAÇÃO QUALITATIVA ENTRE TOPOLOGIAS (N nós)

| Topologia                               | Diâmetro (D)              | Largura da Bissecção (B)      | Grau do Nó (g)                                                       | Custo (nº de enlaces) |
|-----------------------------------------|---------------------------|-------------------------------|----------------------------------------------------------------------|-----------------------|
| Barramento                              | $D = N - 1$               | $B = 1$                       | $g_{\text{extremo}} = 1, g_{\text{interno}} = 2$                     | $C = N - 1$           |
| Anel                                    | $D = \lfloor N/2 \rfloor$ | $B = 2$                       | $g = 2$                                                              | $C = N$               |
| Malha 2D ( $\sqrt{N} \times \sqrt{N}$ ) | $D = 2(\sqrt{N} - 1)$     | $B = \sqrt{N}$                | $g_{\text{canto}} = 2, g_{\text{borda}} = 3, g_{\text{interno}} = 4$ | $C = 2N - 2\sqrt{N}$  |
| Cubo n-D ( $N = 2^n$ )                  | $D = n = \log_2 N$        | $B = N/2$                     | $g = n = \log_2 N$                                                   | $C = nN/2$            |
| Rede Completa (Clique)                  | $D = 1$                   | $B = \lfloor N^2 / 4 \rfloor$ | $g = N - 1$                                                          | $C = N(N - 1)/2$      |

### LEGENDA

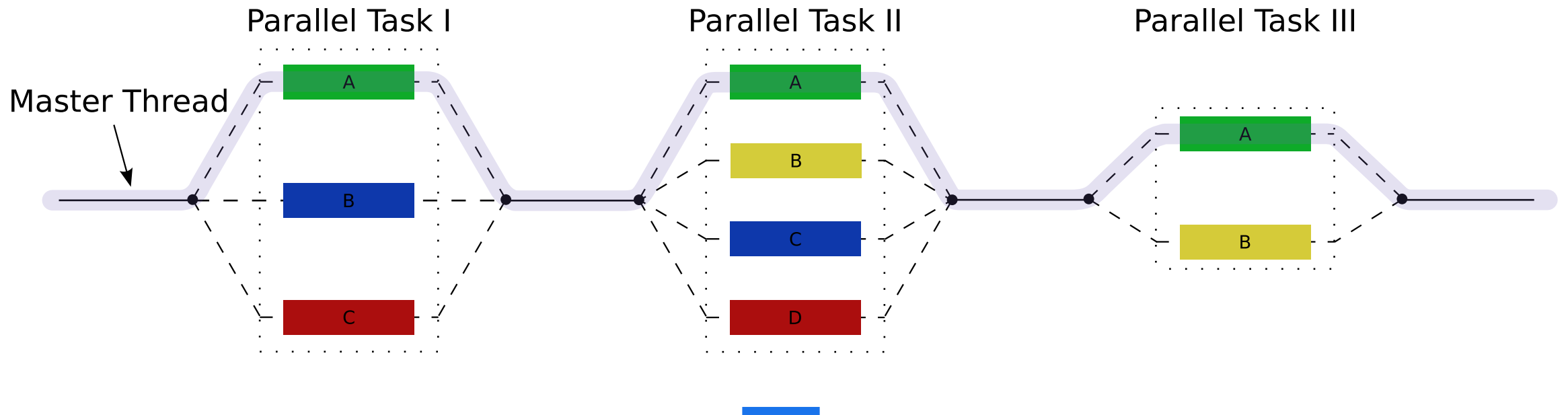
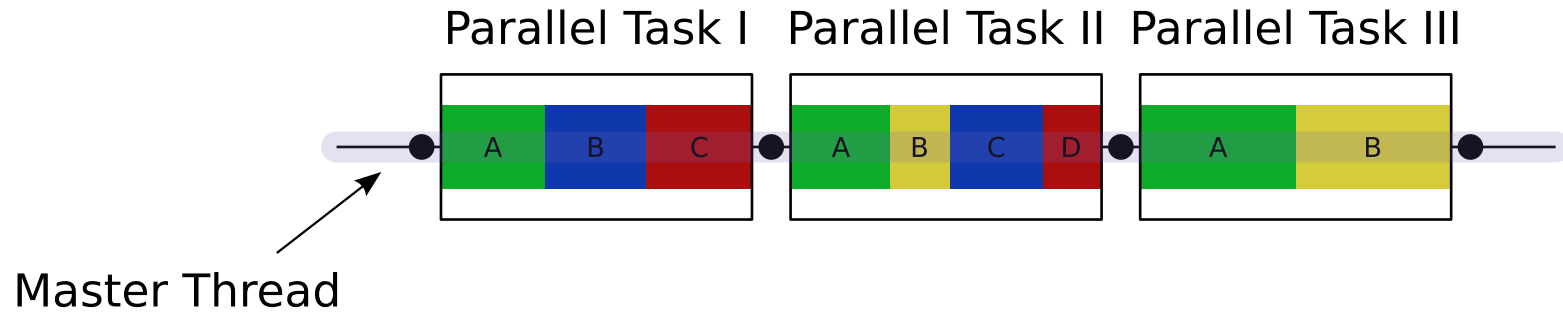
-  Nó (processador)
-  Enlace de comunicação
-  Linha de bissecção
-  Enlace removido
-  Nó de origem/destino

### RESUMO RÁPIDO

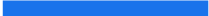
-  **Menor diâmetro** → menor latência máxima
-  **Maior bissecção** → maior banda agregada entre partes da rede
-  **Maior grau** → mais conexões por nó, mais custo e complexidade
-  **Menor custo** → menos enlaces/switches, menor investimento



# Execução em Sistemas Paralelos



Concorrência x Paralelismo



# Paralelismo Implícito x Explícito

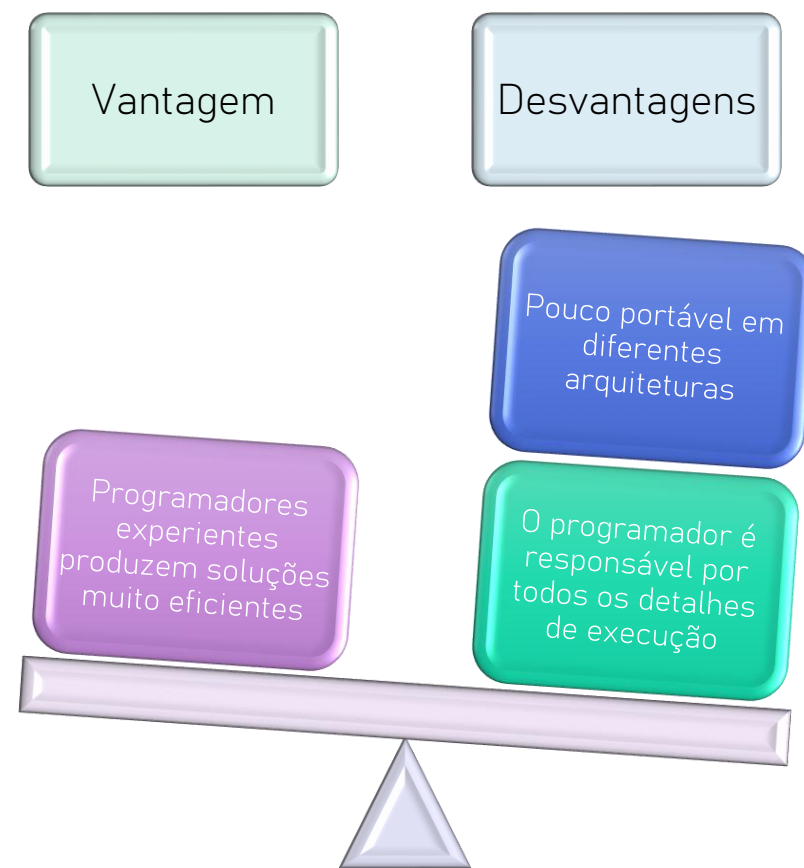
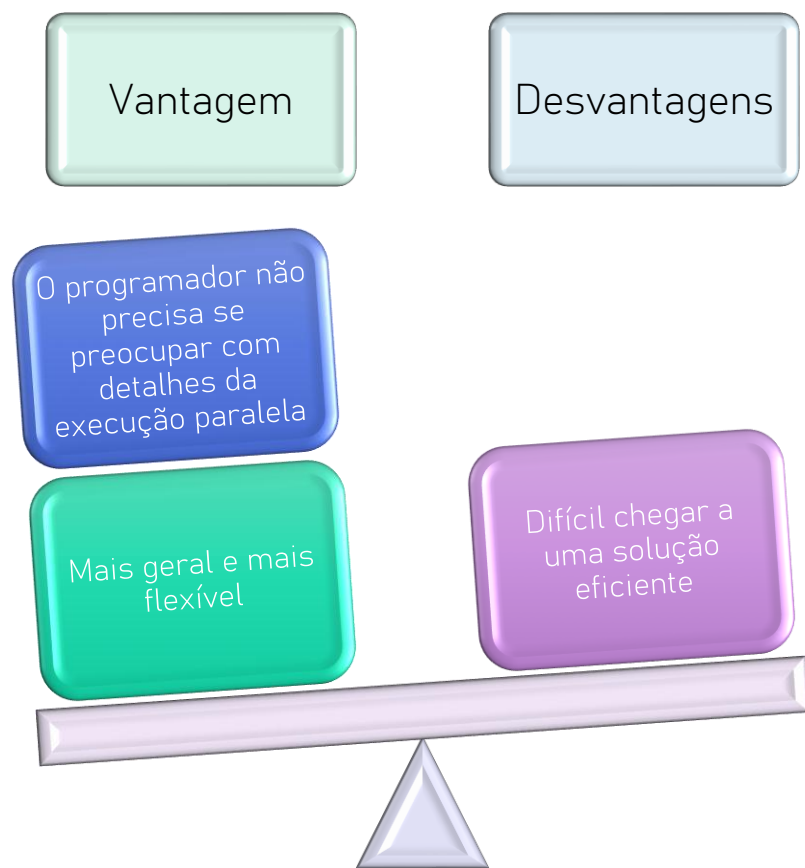
## QUANDO É DE RESPONSABILIDADE DO COMPILADOR E DO SISTEMA DE EXECUÇÃO:

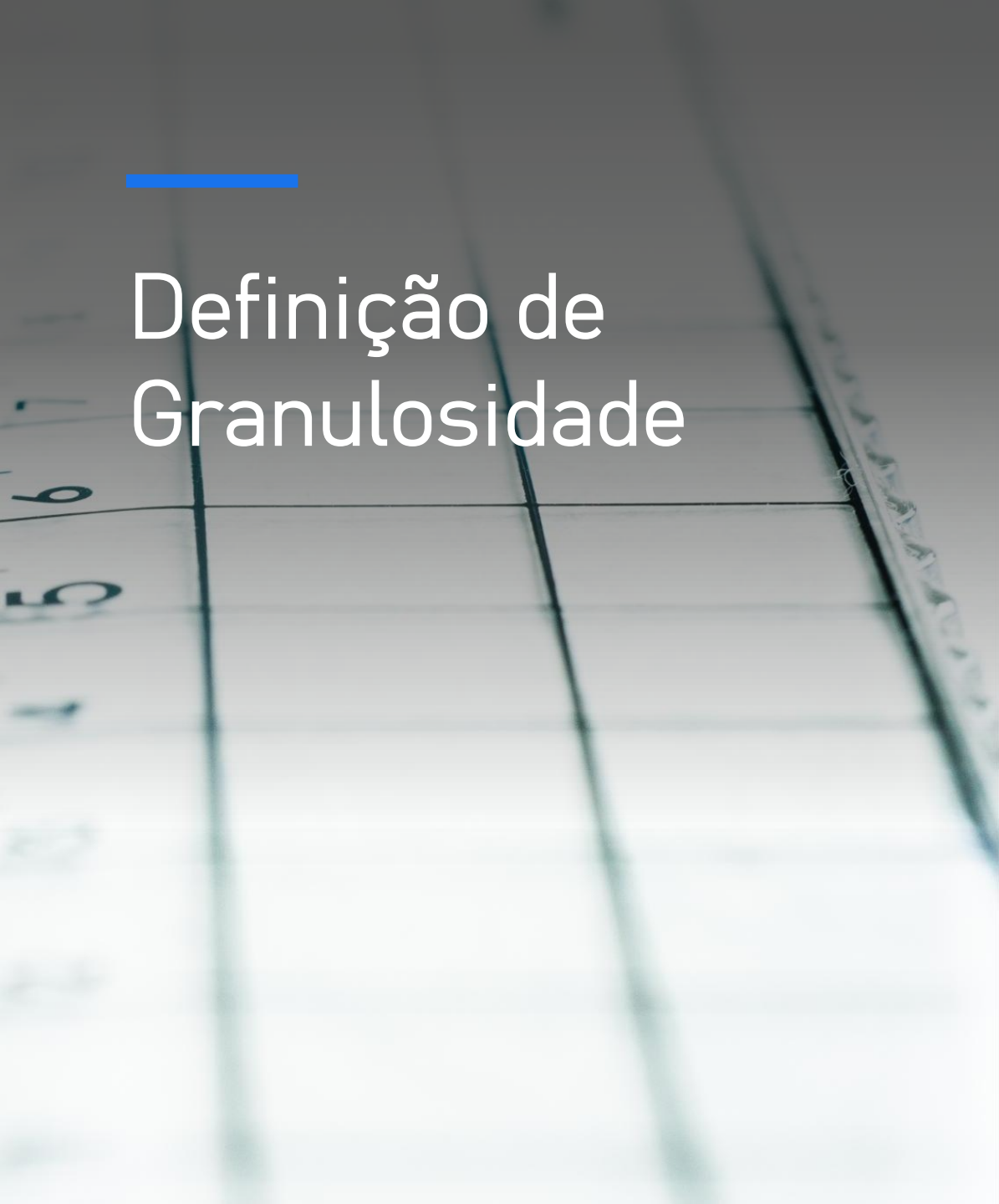
- Detectar o paralelismo do programa;
- Atribuir as tarefas para execução paralela;
- Controlar e sincronizar toda a execução.

## QUANDO É RESPONSABILIDADE DO PROGRAMADOR

- Detectar o paralelismo do programa;
- Atribuir as tarefas aos processadores;
- Controlar a execução, indicando os pontos de sincronização;
- Conhecer a arquitetura do computador, de forma a otimizar o desempenho

# Paralelismo Implícito x Explícito





# Definição de Granulosidade

- Granulosidade refere-se ao tamanho das tarefas e comunicação entre elas.
- É a proporção entre tempo de cálculo e comunicação.
- Determina eficiência do paralelismo e sincronização necessária.

# Granulosidade Fina: Características e Exemplos



- Tarefas pequenas que exigem muita comunicação.
- Exemplo: cada thread processa 1 pixel de uma imagem.
- Permite paralelismo massivo, ideal para GPUs.
- Desvantagem: alto custo de sincronização pode superar ganhos.

# Granulosidade Grossa: Vantagens e Desvantagens



- Tarefas maiores com pouca comunicação.
- Exemplo: cada thread processa blocos de 10.000 pixels.
- Reduz a sobrecarga de comunicação.
- Pode causar desbalanceamento de carga entre tarefas.

## Granulosidade Média: Equilíbrio entre Desempenho e Comunicação



- Compromisso entre granulosidade fina e grossa.
- Buscada na maioria das aplicações práticas.
- Equilibra comunicação e paralelismo.
- Melhora o desempenho geral do sistema.

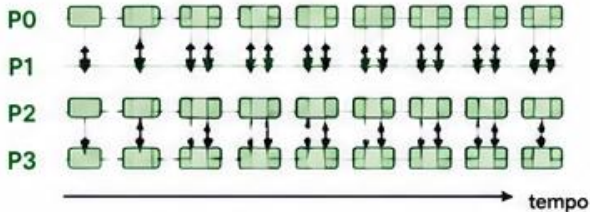
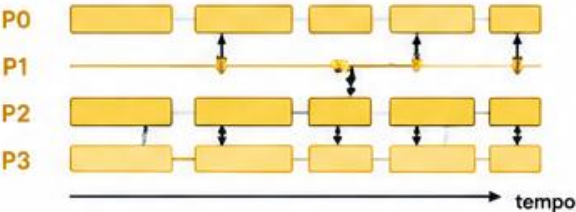
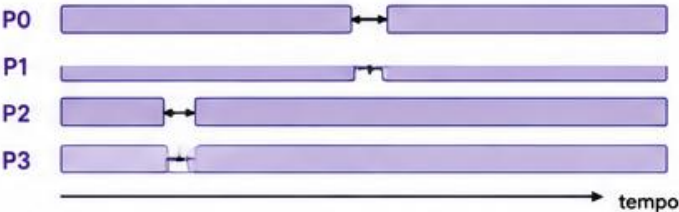
## Exemplo Ilustrativo: Granulosidade no Processamento de Imagens

- Granulosidade fina: 1 pixel por núcleo, muita sincronização.
- Granulosidade grossa:  $1/4$  da imagem por núcleo, pouca comunicação.
- Granulosidade média: blocos menores para equilíbrio.
- Escolha depende do hardware e aplicação.



# GRANULOSIDADE: BAIXA x MÉDIA x ALTA

Comparação dos níveis de granulosidade em aplicações paralelas

| ASPECTO                        | BAIXA GRANULOSIDADE<br>(Fina)                                                                                                                                                                                                                                            | MÉDIA GRANULOSIDADE<br>(Média)                                                                                                                                                                                                                                          | ALTA GRANULOSIDADE<br>(Grosseira)                                                                                                                                                                                                                                                             |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DEFINIÇÃO                      | As tarefas são <b>muito pequenas</b> .<br>Há <b>grande frequência de comunicação</b> entre os processadores.                                                                                                                                                             | As tarefas têm <b>tamanho moderado</b> .<br>Há um <b>equilíbrio</b> entre comunicação e computação.                                                                                                                                                                     | As tarefas são <b>muito grandes</b> .<br>Há <b>pouca comunicação</b> entre os processadores.                                                                                                                                                                                                  |
| EXEMPLO DE APLICAÇÃO           | Ex.: Soma de vetores com poucos elementos, atualização de células em PDE pequena, algoritmos iterativos com muitas sincronizações.                                                                                                                                       | Ex.: Multiplicação de matrizes de tamanho médio, simulações numéricas 2D/3D moderadas, processamento de imagens em blocos.                                                                                                                                              | Ex.: Simulações de grandes fenômenos, análise de grandes conjuntos de dados, tarefas independentes e longas.                                                                                                                                                                                  |
| DIAGRAMA<br>(Visão Conceitual) | <p><b>MUITAS TAREFAS PEQUENAS</b></p>                                                                                                                                                   | <p><b>TAREFAS DE TAMANHO MODERADO</b></p>                                                                                                                                            | <p><b>POUCAS TAREFAS GRANDES</b></p>                                                                                                                                                                       |
| CARACTERÍSTICAS                | <ul style="list-style-type: none"><li>• Tarefas muito pequenas</li><li>• Muitas mensagens/sincronizações</li><li>• Alta dependência entre tarefas</li><li>• Balanceamento geralmente fácil</li></ul>                                                                     | <ul style="list-style-type: none"><li>• Tarefas de tamanho intermediário</li><li>• Número moderado de mensagens</li><li>• Dependência moderada entre tarefas</li><li>• Balanceamento geralmente bom</li></ul>                                                           | <ul style="list-style-type: none"><li>• Tarefas muito grandes</li><li>• Poucas mensagens/sincronizações</li><li>• Baixa dependência entre tarefas</li><li>• Balanceamento pode ser difícil</li></ul>                                                                                          |
| VANTAGENS                      |  <ul style="list-style-type: none"><li>• Boa utilização dos processadores (se bem balanceado)</li><li>• Resultados podem ser obtidos rapidamente (se comunicação for rápida)</li></ul> |  <ul style="list-style-type: none"><li>• Bom compromisso entre comunicação e computação</li><li>• Escalabilidade razoável</li><li>• Melhor aproveitamento dos recursos</li></ul>    |  <ul style="list-style-type: none"><li>• Baixa sobrecarga de comunicação</li><li>• Escalabilidade potencialmente alta</li><li>• Melhor desempenho por tarefa</li></ul>                                    |
| DESVANTAGENS                   |  <ul style="list-style-type: none"><li>• Sobrecarga de comunicação muito alta</li><li>• Escalabilidade limitada</li><li>• Sensível à latência da rede e sincronizações</li></ul>      |  <ul style="list-style-type: none"><li>• Pode tornar-se comunicação limitada se as tarefas ficarem pequenas demais</li><li>• Requer análise para encontrar o ponto ótimo</li></ul> |  <ul style="list-style-type: none"><li>• Balanceamento pode ser difícil (tarefa muito grande)</li><li>• Menor interação entre processadores</li><li>• Alguns processadores podem ficar ociosos</li></ul> |
| RELAÇÃO COMUNICAÇÃO/COMPUTAÇÃO |  <p><b>Comunicação &gt;&gt; Computação</b><br/>Razão muito alta (desfavorável)</p>                                                                                                    |  <p><b>Comunicação ≈ Computação</b><br/>Razão equilibrada (ideal)</p>                                                                                                              |  <p><b>Comunicação &lt;&lt; Computação</b><br/>Razão muito baixa (desfavorável se exagerada)</p>                                                                                                         |

# GRANULOSIDADE: BAIXA x MÉDIA x ALTA

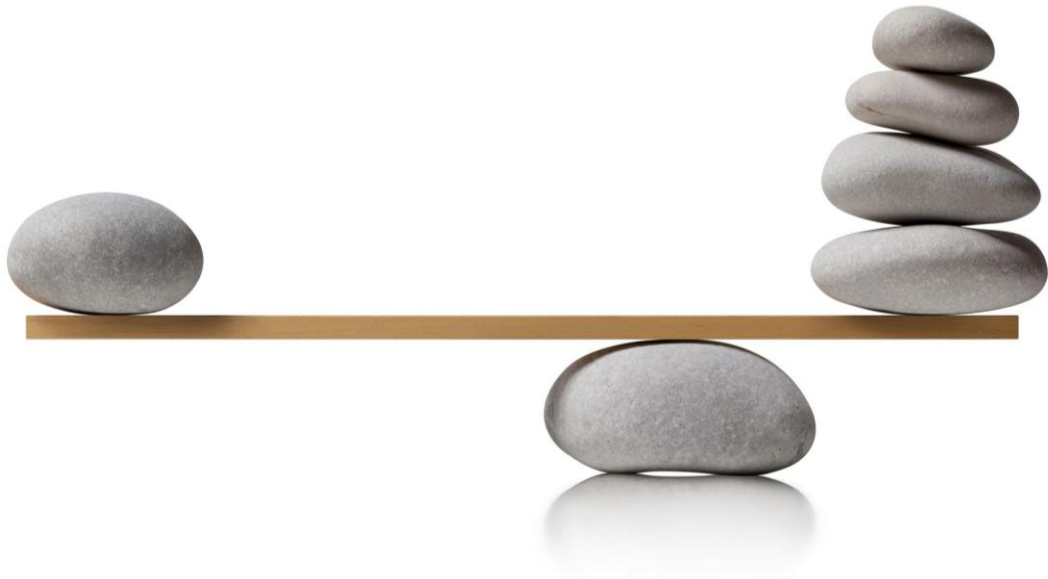
Comparação dos níveis de granulosidade em aplicações paralelas

| <div>RESUMO COMPARATIVO</div> <div></div> |  Granulosidade |  Tamanho da Tarefa |  Frequência de Comunicação |  Overhead de Comunicação |  Escalabilidade |  Grau de Desbalanceamento | Exemplos Típicos                                                 |
|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
|                                                                                                                           | Baixa (Fina)                                                                                    | Muito pequena                                                                                       | Muito alta                                                                                                  | Muito alto                                                                                                  | Limitada                                                                                           | Baixo                                                                                                        | Atualizações finas, métodos iterativos com muitas sincronizações |
|                                                                                                                           | Média (Média)                                                                                   | Intermediária                                                                                       | Moderada                                                                                                    | Moderado                                                                                                    | Razoável                                                                                           | Médio                                                                                                        | Multiplicação de matrizes, simulações 2D/3D médias               |
|                                                                                                                           | Alta (Grosseira)                                                                                | Muito grande                                                                                        | Muito baixa                                                                                                 | Muito baixo                                                                                                 | Alta (potencial)                                                                                   | Alto                                                                                                         | Simulações globais, análise de grandes volumes de dados          |



Escolher o nível de granulosidade ideal depende do problema, da arquitetura e dos objetivos: buscar o melhor equilíbrio entre comunicação e computação.

# Balanceamento de Carga no Paralelismo



- Distribuição equilibrada do trabalho entre processadores ou núcleos.
- Evita sobrecarga em alguns processadores enquanto outros ficam ociosos.
- Busca maximizar eficiência e reduzir o tempo total de execução.



# O que é Balanceamento de Carga?

É a técnica de distribuir tarefas, dados ou requisições entre múltiplos processadores, núcleos ou nós de um sistema, de forma que nenhum recurso fique sobrecarregado enquanto outros permanecem ociosos.

O objetivo é maximizar o uso dos recursos, reduzir o tempo total de execução e evitar gargalos.



## **Eficiência**

Uso equilibrado de CPU, memória e rede em todo o sistema



## **Desempenho**

Redução do tempo total de execução das aplicações



## **Escalabilidade**

Sistemas que crescem sem criar novos gargalos

---

# Importância do Balanceamento de Carga

- Processadores ociosos aumentam o tempo total de execução.
- Processador mais lento determina o tempo final do sistema.
- Um bom balanceamento resulta em maior speedup e eficiência.



# Tipos de Balanceamento de Carga

- Estático: carga distribuída antes da execução, simples porém inflexível.
- Dinâmico: ajusta a distribuição durante a execução, adaptando-se a cargas desiguais.
- Cada tipo possui vantagens e desvantagens específicas.

# Duas Abordagens Principais



## ESTÁTICO

*decisão a priori*

A distribuição das tarefas é definida antes da execução, com base em informações conhecidas previamente — número de processadores e tamanho estimado das tarefas. Não considera o estado real do sistema em tempo de execução.

---

Baixo overhead • Simples de implementar • Ideal para cargas previsíveis



## DINÂMICO

*decisão em runtime*

As tarefas são redistribuídas durante a execução, com base no estado atual do sistema — carga de cada nó, tarefas concluídas. Exige comunicação e monitoramento contínuos entre os processos.

---

Maior overhead • Mais complexo • Ideal para cargas imprevisíveis

---

# Balanceamento Estático

## Vantagens

Simple de implementar.

Menor sobrecarga de comunicação e coordenação.

Previsível: já se sabe quanto trabalho cada processador terá.

## Desvantagens

Rigidez: não se adapta a variações dinâmicas de carga.

Pode levar a desbalanceamento se o trabalho não for homogêneo.

Mau aproveitamento em ambientes heterogêneos (processadores com capacidades diferentes).



# Balanceamento Estático — Variantes

*Estratégias definidas antes da execução*



## Round Robin

Tarefas distribuídas ciclicamente, uma a uma, entre os processadores disponíveis.



## Randomizado

Cada tarefa é atribuída a um processador escolhido aleatoriamente.



## Baseado em Limiar

Atribuição segundo regras e limites pré-definidos de capacidade.



## Particionamento por Tamanho

Tarefas maiores e menores são combinadas para equilibrar a carga total.

---

# Balanceamento Dinâmico

## Vantagens

Flexível: ajusta-se em tempo de execução.

Melhor desempenho quando as tarefas têm cargas desiguais.

Adapta-se a falhas ou diferenças de desempenho entre processadores.

## Desvantagens

Maior sobrecarga de comunicação e sincronização.

Implementação mais complexa.

Possível latência ao redistribuir tarefas.



# Balanceamento Dinâmico — Variantes

*Estratégias que reagem ao estado do sistema em tempo real*



## Centralizado

Um nó coordenador monitora a carga e decide a distribuição das tarefas.



## Distribuído

Cada nó decide localmente, trocando informações com seus vizinhos.



## Hierárquico

Combina níveis de decisão local e global em uma estrutura em camadas.



## Work Stealing

Processos ociosos "roubam" tarefas da fila de processos ocupados.

# Comparação entre as Abordagens

## ESTÁTICO

-  **Overhead**  
Baixo ou inexistente
-  **Complexidade**  
Simples de implementar
-  **Adaptação a mudanças**  
Não se adapta durante a execução
-  **Tolerância a falhas**  
Baixa — um nó lento atrasa o todo
-  **Escalabilidade**  
Boa em sistemas pequenos e estáveis
-  **Cenário ideal**  
Clusters homogêneos, cargas previsíveis

## DINÂMICO

-  **Overhead**  
Moderado a alto
-  **Complexidade**  
Requer sincronização e migração de tarefas
-  **Adaptação a mudanças**  
Adapta-se continuamente
-  **Tolerância a falhas**  
Melhor — redistribui em caso de falha
-  **Escalabilidade**  
Melhor em sistemas grandes e heterogêneos
-  **Cenário ideal**  
Nuvem, grades computacionais, cargas variáveis

# Quando Usar

---



## Prefira o Estático quando...

- A carga de trabalho é homogênea e previsível
- O ambiente é estável, sem variações de desempenho
- Baixo overhead é prioridade

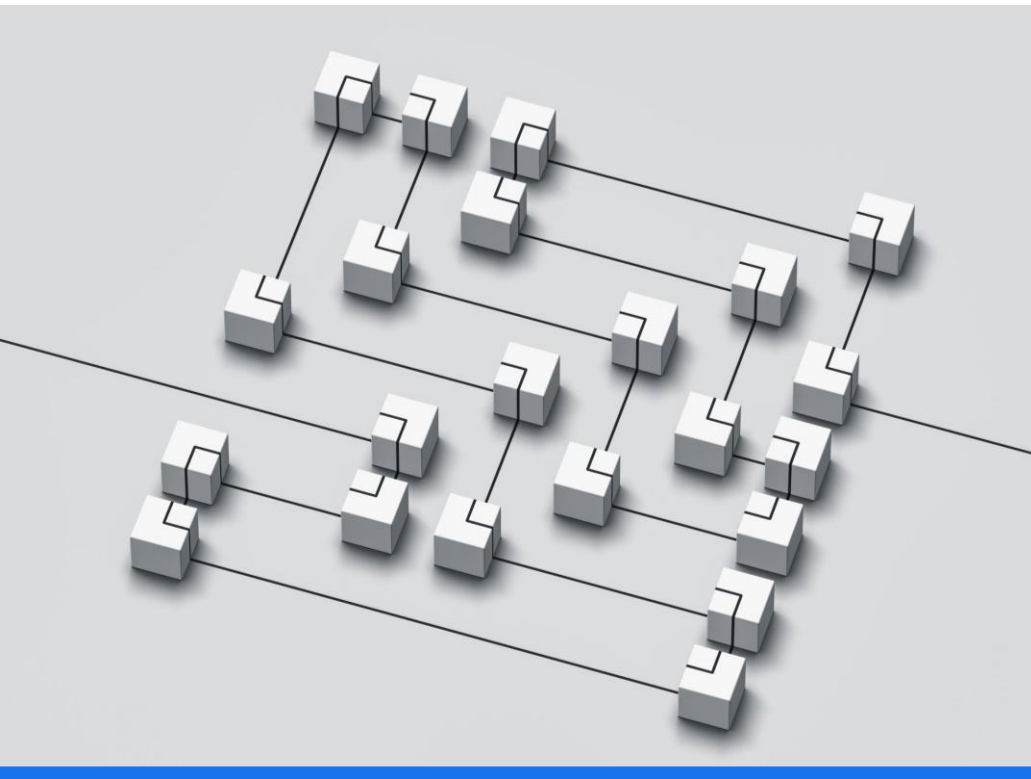


## Prefira o Dinâmico quando...

- A carga é heterogênea ou imprevisível
- O sistema é grande, distribuído ou baseado em nuvem
- Tolerância a falhas é essencial

*Na prática, muitos sistemas combinam as duas abordagens — uma distribuição inicial estática seguida de ajustes dinâmicos ao longo da execução.*

# Conclusão



- A computação paralela surge como resposta essencial aos limites da computação sequencial, permitindo dividir problemas complexos em tarefas menores executadas simultaneamente.
- **Tipos e arquiteturas** (Flynn, memória compartilhada/distribuída) definem *como* o paralelismo é estruturado.
- **Redes de interconexão** determinam a eficiência da comunicação entre processadores.
- **Granulosidade e balanceamento de carga** são decisivos para transformar paralelismo em ganho real de desempenho.
- O desafio central não é apenas paralelizar, mas equilibrar comunicação, sincronização e distribuição de trabalho — tema que será aprofundado nas próximas aulas, com modelos de programação e métricas de desempenho (speedup, eficiência, Lei de Amdahl).